

Systematische Migration von
Excel-Arbeitsmappen und VBA-Logik
nach PostgreSQL und Python

Stephan Epp

14. März 2026

Inhaltsverzeichnis

1	Einleitung	4
2	Systemarchitektur	6
2.1	Überblick der Zielarchitektur	6
2.2	Migrationsäquivalenzen	7
3	Entwurf: Datenbankschema	9
3.1	Normalisierungsstrategie	9
3.2	Konkretes Datenbankschema	9
3.3	Entity-Relationship-Diagramm	11
4	ETL-Pipeline: Excel → PostgreSQL	12
4.1	Pipeline-Übersicht	12
4.2	Vollständiger ETL-Python-Code	12
5	VBA-zu-Python-Äquivalenz	17
5.1	Iteration über Zeilen	17
5.2	Datei öffnen und speichern	18
5.3	Bedingte Formatierung / Hervorhebung	18
6	Analytische SQL-Abfragen	20
7	Migrationsprozess: Schritt für Schritt	22
8	Vergleich: Legacy vs. Zielsystem	24
9	Versionskontrolle mit Git: Der entscheidende Strukturvorteil	25
9.1	Das Versionierungsproblem von Excel	25
9.2	Repository-Struktur – BioFalcon-Migrationsprojekt	26
9.3	Grundlegende Git-Kommandos im Migrationskontext	28
9.4	Schema-Migrationen versionieren	29
9.5	Branching-Strategie für sichere Weiterentwicklung	30
9.6	Rollback: Änderungen rückgängig machen	31
9.7	Continuous Integration: Automatisierte Qualitätssicherung	31
9.8	Git-Prüfpfad als Audit Trail	33
9.9	Pre-Commit-Hooks: Qualität vor dem Commit erzwingen	33
9.10	Zusammenfassung: Quantitiver Vorteil von Git	34

10 Produktive Bereitstellung: Asynchrone Datenbankschicht	35
10.1 Motivation: Warum asyncpg statt psycopg2?	35
10.2 Produktiver asyncpg-Migrationsclient	36
10.3 Transaktionssicherheit und Rollback	40
11 Pipeline-Orchestrierung: Airflow und Prefect	41
11.1 Übersicht: Wann welches Tool?	41
11.2 Apache Airflow: DAG-Definition	42
11.3 Prefect: Alternative mit deklarativer Flow-Syntax	44
12 Containerisierung und Deployment-Infrastruktur	46
12.1 Docker-Compose-Stack	46
12.2 Dockerfile für die ETL-Applikation	48
13 Sicherheitsarchitektur	50
13.1 Credential-Management	50
13.2 Minimale PostgreSQL-Benutzerrechte	51
14 Monitoring und Observability	53
14.1 Prometheus-Metriken in der ETL-Pipeline	53
14.2 Grafana-Dashboard-Konfiguration	54
15 Testarchitektur	56
15.1 Teststrategie für ETL-Pipelines	56
16 Vollständiges Produktiv-Deployment-Playbook	59
16.1 Voraussetzungen und Verzeichnisstruktur	59
16.2 Schritt 1: Umgebungsvariablen konfigurieren	59
16.3 Schritt 2: Stack starten und Schema initialisieren	60
16.4 Schritt 3: Erstimport ausführen	60
16.5 Schritt 4: Airflow aktivieren und DAG triggern	61
16.6 Schritt 5: Monitoring verifizieren	61
16.7 Schritt 6: Rollback-Strategie	62
16.8 Schritt 7: Inkrementelle Synchronisation (Deltas)	62
17 Schluss	64
17.1 Technische Weiterentwicklung der Migrationsarchitektur	64
17.2 Konsequenzen für die Wirtschaft und das Management	65
17.2.1 Kostensenkung durch Eliminierung von Fehlerquellen	65
17.2.2 Skalierbarkeit als Wettbewerbsvorteil	65
17.2.3 Reduktion von Knowledge-Lock-in	66
17.3 Konsequenzen für die Industrieproduktion und Fertigung	66
17.3.1 Echtzeit-Qualitätssicherung (SPC)	66
17.3.2 Rückverfolgbarkeit (Traceability) in der normkonformen Fertigung	66
17.3.3 Predictive Maintenance und IIoT-Integration	66
17.4 Konsequenzen für das Finanzwesen	67
17.4.1 Regulatorische Compliance und Basel III/IV	67
17.4.2 Risikomanagement und Real-Time-Pricing	67
17.4.3 Automatisierte Berichterstattung und Regulatorik	67

17.5	Konsequenzen für die wissenschaftliche Forschung	68
17.5.1	Reproduzierbarkeit als Wissenschaftsnorm (FAIR-Prinzip)	68
17.5.2	Kollaborative Forschung und Open Science	68
17.5.3	Meta-Analysen, Datenfusion und Forschung	68
17.5.4	Langzeitarchivierung von Forschungsdaten	69
17.6	Konsequenzen für das Bildungswesen und die Lehre	69
17.6.1	Datenkompetenz als Kernkompetenz des 21. Jahrhunderts	69
17.6.2	Strukturierte Lehre mit Git und Continuous Integration	69
17.7	Konsequenzen für das Gesundheitswesen und die Medizin	69
17.7.1	Klinische Studien und GCP-Compliance	70
17.7.2	Epidemiologie, Surveillance, Echtzeit-Pandemiemonitoring	70
17.8	Konsequenzen für Behörden und Verwaltung	70
17.8.1	E-Government und Open Data	70
17.8.2	Bekämpfung von Haushaltsbetrug und Korruption	70
17.9	Konsequenzen für Nachhaltigkeit und Klimaforschung	71
17.9.1	Treibhausgas-Bilanzierung und ESG-Reporting	71
17.9.2	Klimamodellierung und Ensemble-Berechnungen	71
17.10	Gesellschaftliche Dimension: Datensouveränität	71
17.10.1	Vendor-Lock-in und digitale Souveränität	71
17.10.2	Datenschutz (DSGVO) und Datensicherheit	71
17.11	Gesamtfazit: Ein Paradigmenwechsel mit globalem Horizont	72
17.12	Handlungsempfehlungen	73

Kapitel 1

Einleitung

Die vorliegende Arbeit beschreibt eine vollständige, reproduzierbare Migrationsstrategie zur Ablösung von Microsoft-Excel-Arbeitsmappen (inklusive VBA-Makroautomatisierung) durch ein modernes, datenbankgestütztes Python-Ökosystem auf Basis von PostgreSQL als persistenter Datenschicht. Neben der konzeptionellen ETL-Architektur und dem normierten Datenbankschema wird der Schwerpunkt auf die **produktive Bereitstellung** gelegt: asynchrone Datenbankzugriffe via `asyncpg`, Pipeline-Orchestrierung mit Apache Airflow und Prefect, Containerisierung via Docker, Monitoring, Sicherheitsarchitektur, Rollback-Strategie sowie ein vollständiges Deployment-Playbook. Der Ansatz ist exemplarisch am Windkanal-Datensatz des BioFalcon-Projekts demonstriert, aber generisch auf beliebige Excel-basierte Ingenieursworkflows übertragbar.

De-facto-Plattform für Datenhaltung, Berechnung und Automatisierung etabliert. Insbesondere in Bereichen wie Aerodynamik, Regelungstechnik und eingebetteten Systemen werden Mess- und Parametertabellen häufig als `.xlsx`-Dateien gepflegt, während VBA-Makros die Prozesslogik kodieren.

Dieser Ansatz ist mit strukturellen Nachteilen verbunden:

- **Keine echte Datenbankintegrität:** Excel kennt weder Fremdschlüssel noch Transaktionssicherheit. Redundanz und Inkonsistenz entstehen zwangsläufig bei wachsenden Datensätzen.
- **Mangelnde Versionierbarkeit:** Binäre `.xlsx`-Dateien lassen sich mit Git nur eingeschränkt verwalten.
- **Sprachlicher Lock-in durch VBA:** VBA ist proprietär, schlecht testbar und nicht kompatibel mit modernen Continuous-Integration-Pipelines.
- **Eingeschränkte Skalierbarkeit:** Excel ist bei großen Datensätzen (> 100 000 Zeilen) signifikant langsamer als PostgreSQL mit Indexstrukturen.
- **Fehlende Reproduzierbarkeit:** Berechnungen in Zellen sind schwer zu dokumentieren und zu auditieren.

Python in Kombination mit PostgreSQL löst diese Probleme systematisch: `pandas` übernimmt die Datenmanipulation, `SQLAlchemy` die datenbankagnostische Verbindungsschicht, und `psycopg2` den nativen PostgreSQL-Treiber.

Ausgangssituation des Anwenders Der Anwender besitzt zu Beginn der Migration ausschließlich:

1. Den Python-Quellcode des Migrationsskripts (`migrate.py`)
2. Eine laufende PostgreSQL-Instanz (leer, mit Zugangsdaten)

Die Excel-Quelldatei wird durch synthetische Testdaten simuliert. Im Produktiveinsatz wird `migrate.py` direkt gegen die reale `.xlsx`-Datei ausgeführt.

Kapitel 2

Systemarchitektur

Dieses Kapitel beschreibt die Zielarchitektur des migrierten Systems und legt damit das konzeptionelle Fundament für alle nachfolgenden Implementierungsentscheidungen. Ausgangspunkt ist die klassische dreischichtige Softwarearchitektur, die für den spezifischen Kontext einer **Excel+VBA**-Migration nach **PostgreSQL+Python** konkretisiert wird. Im Mittelpunkt steht die Frage, welche Systemkomponenten miteinander interagieren, welche Schnittstellen zwischen ihnen bestehen und wie Datenflüsse organisiert sind. Darüber hinaus werden die funktionalen Äquivalenzen zwischen Legacy-**Excel**-Konstrukten und modernen Python/**PostgreSQL**-Pendants systematisch gegenübergestellt, sodass jede Migrationsentscheidung nachvollziehbar und begründbar bleibt.

2.1 Überblick der Zielarchitektur

Die Zielarchitektur folgt dem klassischen dreischichtigen Modell: Datenschicht (**PostgreSQL**), Logikschicht (**Python**) und Präsentationsschicht (CLI / Visualisierung). Figure [2.1](#) zeigt das vollständige Komponentendiagramm.

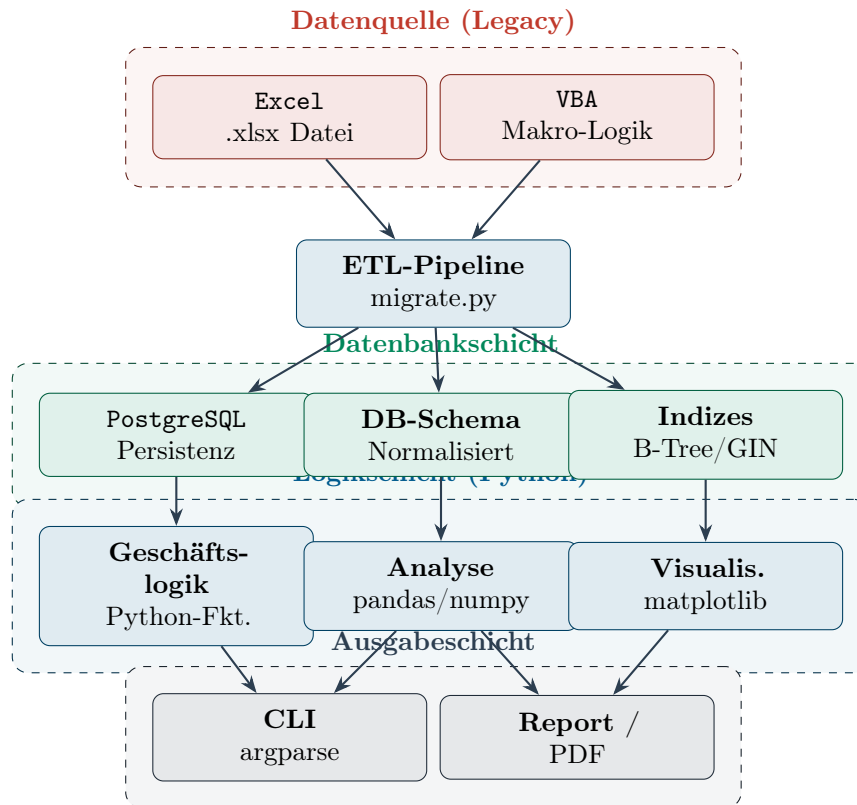


Abbildung 2.1: Zielarchitektur: Migration von Excel+VBA nach PostgreSQL+Python

2.2 Migrationsäquivalenzen

Table 2.1 listet die vollständigen funktionalen Äquivalenzen zwischen dem Legacy-System (Excel+VBA) und dem Zielsystem (PostgreSQL+Python).

Tabelle 2.1: Funktionale Äquivalenzen: Excel+VBA $\rightarrow$ PostgreSQL+Python

Excel / VBA	Funktion	Python / PostgreSQL
Tabellenblatt	Datentabelle	pg: Relation (TABLE)
Zellbereich / named range	Strukturierte Selektion	pd.DataFrame / SQL SELECT
Formel =E2/F2	Berechnete Spalte	df["L_D"] = df["C_L"]/df["C_D"]
WENN()-Formel	Konditionale Logik	df.apply() / CASE WHEN
VBA Sub / Function	Prozedurale Einheit	Python def
VBA For Each ... Next	Iteration über Zeilen	df.iterrows() / SQL CURSOR
VBA Workbooks.Open	Datei öffnen	pd.read_excel()
VBA ActiveSheet.SaveAs	Datei speichern	df.to_csv() / to_excel()
VBA MsgBox / Debug.Print	Ausgabe	print() / logging
Bedingte Formatierung	Visuelle Hervorhebung	matplotlib color mapping
Pivot-Tabelle	Aggregation	df.groupby().agg() / GROUP BY
Excel-Diagramm	Visualisierung	matplotlib / plotly

Kapitel 3

Entwurf: Datenbankschema

Ein sorgfältig entworfenes Datenbankschema ist das Herzstück jeder produktiven Datenmigration. Während **Excel**-Arbeitsmappen Daten häufig in einer flachen, denormalisierten Tabellenstruktur vorhalten, erfordert ein relationales Datenbanksystem wie **PostgreSQL** eine explizite Modellierung von Entitäten, Attributen und ihren Beziehungen untereinander. Dieses Kapitel dokumentiert den Entwurfsprozess des Zielschemas Schritt für Schritt: von der Analyse der Ausgangsdaten über die Ableitung einer normalisierten Relationenstruktur bis hin zum finalen SQL-DDL-Code sowie der grafischen Darstellung im Entity-Relationship-Diagramm. Besonderes Augenmerk liegt dabei auf der Wahrung von referenzieller Integrität, der Vergabe geeigneter Datentypen und der Vorbereitung des Schemas für spätere Erweiterungen im Produktivbetrieb.

3.1 Normalisierungsstrategie

Excel-Tabellen liegen typischerweise in denormalisierter Form vor (Erste Normalform oder schlechter). Der Migrationsprozess überführt die Daten in mindestens die **Dritte Normalform (3NF)**.

Definition 3.1.1 (Migrationsnormalisierung). Sei T eine **Excel**-Tabelle mit Spaltenmengen C und Zeilenmenge R . Die Zielnormalisierung $\mathcal{N}(T)$ erzeugt eine Menge von Relationen $\{R_1, \dots, R_k\}$ in 3NF, sodass gilt:

$$\forall i \neq j : R_i \cap R_j \subseteq \{\text{Primärschlüssel}\} \quad \text{und} \quad \bigcup_{i=1}^k \pi_C(R_i) \supseteq C$$

d. h. alle ursprünglichen Spalten sind verlustfrei rekonstruierbar.

3.2 Konkretes Datenbankschema

Das folgende DDL-Statement erzeugt das normalisierte Schema für den BioFalcon-Anwendungsfall. Es ist direkt auf beliebige **Excel**-Datensätze verallgemeinerbar.

```
1  -- Schema-Initialisierung
2  CREATE SCHEMA IF NOT EXISTS biofalcon;
3  SET search_path TO biofalcon;
4
```

```

5  -- Lookup-Tabelle: Str\omungsregime (normalisiert aus Excel-
    Textspalte)
6  CREATE TABLE stroemungsregime (
7      id          SMALLSERIAL PRIMARY KEY,
8      name        VARCHAR(32) NOT NULL UNIQUE,
9      sortkey     SMALLINT    NOT NULL DEFAULT 0
10 );
11
12 INSERT INTO stroemungsregime (name, sortkey) VALUES
13     ('laminar',    1),
14     ('transition', 2),
15     ('turbulent',  3);
16
17 -- Haupttabelle: Messung
18 CREATE TABLE messung (
19     id          BIGSERIAL PRIMARY KEY,
20     messung_id  VARCHAR(8) NOT NULL UNIQUE, -- 'A06'
21     alpha_deg   NUMERIC(6,2) NOT NULL,      -- Anstellwinkel
22     re_mio      NUMERIC(6,3) NOT NULL,      -- Reynolds * 10^6
23     c_l         NUMERIC(8,4) NOT NULL,      -- Auftriebsbeiwert
24     c_d         NUMERIC(8,4) NOT NULL,      --
                Widerstandsbeiwert
25     c_m         NUMERIC(8,4),               -- Momentenbeiwert
26     regime_id   SMALLINT NOT NULL
                REFERENCES stroemungsregime(id),
27     anmerkung   TEXT,
28     is_outlier  BOOLEAN NOT NULL DEFAULT FALSE,
29     is_stall    BOOLEAN NOT NULL DEFAULT FALSE,
30     created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
31     CONSTRAINT ck_alpha CHECK (alpha_deg BETWEEN -90 AND 90),
32     CONSTRAINT ck_re_pos CHECK (re_mio > 0),
33     CONSTRAINT ck_cd_pos CHECK (c_d > 0)
34 );
35
36
37 -- Abgeleitete Gr\o\ss\en (berechnet, nicht gespeichert => VIEW)
38 CREATE VIEW v_messung_computed AS
39 SELECT
40     m.*,
41     r.name AS stroemung,
42     ROUND(m.c_l / NULLIF(m.c_d, 0), 3) AS l_d,
43     ROUND(
44         POWER(m.c_l, 1.5) / NULLIF(m.c_d, 0), 4
45     ) AS cl15_cd,
46     ROUND(
47         2.0 * m.c_l /
48         (pi() * 7.0 * (1.0 + 2.0 / 7.0)), 4
49     ) AS c_di_prandtl -- Prandtl -
                Widerstand
50 FROM messung m
51 JOIN stroemungsregime r ON r.id = m.regime_id;
52
53 -- Indizes fuer analytische Abfragen
54 CREATE INDEX idx_messung_alpha ON messung (alpha_deg);
55 CREATE INDEX idx_messung_regime ON messung (regime_id);
56 CREATE INDEX idx_messung_outlier ON messung (is_outlier)
57     WHERE is_outlier = TRUE;
58
59 -- Optimum-View (ersetzt VBA-Makro 'FindeOptima')

```

```

60 CREATE VIEW v_optima AS
61 SELECT
62     'Geschwindigkeit'::TEXT AS kriterium,
63     messung_id, alpha_deg, l_d AS kennwert
64 FROM v_messung_computed
65 WHERE NOT is_outlier AND NOT is_stall
66 ORDER BY l_d DESC NULLS LAST
67 LIMIT 1
68 UNION ALL
69 SELECT
70     'Ausdauer'::TEXT,
71     messung_id, alpha_deg, cl15_cd
72 FROM v_messung_computed
73 WHERE NOT is_outlier AND NOT is_stall
74 ORDER BY cl15_cd DESC NULLS LAST
75 LIMIT 1;

```

Listing 3.1: PostgreSQL DDL: Vollständiges Datenbankschema

3.3 Entity-Relationship-Diagramm

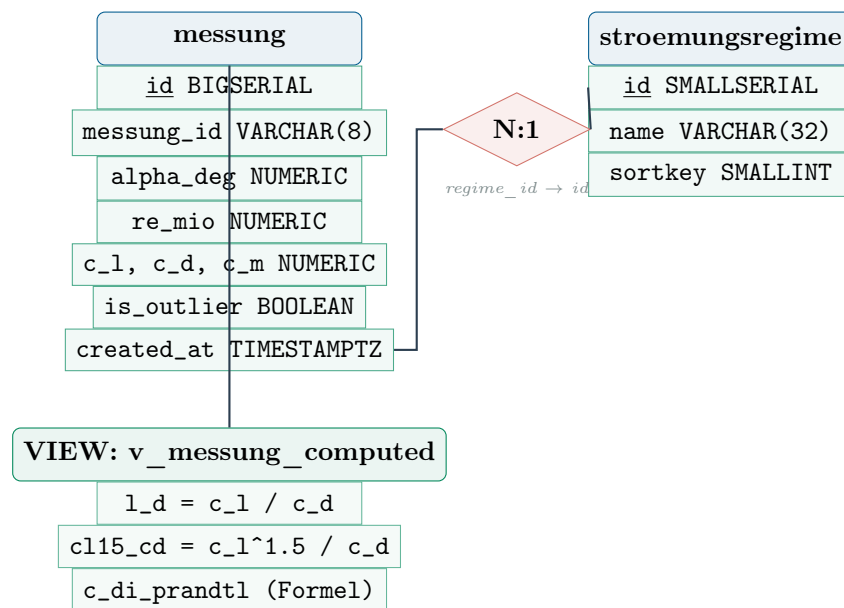


Abbildung 3.1: Entity-Relationship-Diagramm des normierten PostgreSQL-Schemas

Kapitel 4

ETL-Pipeline: Excel $\rightarrow$ PostgreSQL

Die ETL-Pipeline bildet das operative Kernstück der gesamten Migrationslösung. Ihre Aufgabe ist es, rohe **Excel**-Daten zuverlässig, reproduzierbar und fehlertolerant in die normalisierte **PostgreSQL**-Datenbankstruktur zu überführen. Das Akronym ETL steht für die drei Hauptphasen des Prozesses: *Extract* bezeichnet die Extraktion der Quelldaten aus der `.xlsx`-Datei, *Transform* umfasst alle Schritte der Bereinigung, Typkonversion, Normalisierung und Validierung, und *Load* schließlich das atomare Schreiben der transformierten Datensätze in die Datenbank. Dieses Kapitel beschreibt zunächst den konzeptionellen Aufbau der Pipeline, leitet mathematisch das Idempotenz-Theorem her – also die Eigenschaft, dass wiederholte Ausführungen stets dasselbe Ergebnis erzeugen – und stellt anschließend den vollständigen, produktionsreifen Python-Quellcode bereit.

4.1 Pipeline-Übersicht

Die ETL-Pipeline (*Extract – Transform – Load*) besteht aus drei klar abgegrenzten Phasen:

1. **Extract:** Einlesen der **Excel**-Tabellenblätter via `pd.read_excel()` unter Beibehaltung aller Datentypen.
2. **Transform:** Normalisierung, Typkonvertierung, Ableitung fehlender Größen, Plausibilitätsprüfung, Duplikaterkennung.
3. **Load:** Atomares Schreiben in **PostgreSQL** via **SQLAlchemy** mit Upsert-Semantik (`ON CONFLICT DO UPDATE`).

Satz 4.1.1 (ETL-Idempotenz). Sei $\mathcal{E}$ die ETL-Pipeline und D der Quelldatensatz. $\mathcal{E}$ ist idempotent, d. h.:

$$\mathcal{E}(\mathcal{E}(D)) = \mathcal{E}(D)$$

Dies wird durch Upsert-Semantik mit `ON CONFLICT (messung_id) DO UPDATE` garantiert. Mehrfaches Ausführen der Pipeline erzeugt keinen inkonsistenten Datenbankzustand.

4.2 Vollständiger ETL-Python-Code

```

1  #!/usr/bin/env python3
2  """
3  migrate.py -- Excel + VBA -> PostgreSQL + Python
4  Aufruf: python migrate.py --dsn "postgresql://user:pw@host/db"
5          python migrate.py --dsn "." --xlsx messdaten.xlsx
6  """
7
8  import argparse
9  import logging
10 import sys
11 from pathlib import Path
12
13 import pandas as pd
14 from sqlalchemy import create_engine, text
15 from sqlalchemy.exc import SQLAlchemyError
16
17 logging.basicConfig(
18     level=logging.INFO,
19     format="%(asctime)s [%(levelname)s] %(message)s"
20 )
21 log = logging.getLogger(__name__)
22
23
24 # -- 1. EXTRACT
25 -----
26 def extract(xlsx_path: Path) -> dict[str, pd.DataFrame]:
27     """
28     Liest alle Tabellenblätter der Excel-Datei.
29     Entspricht: VBA Workbooks.Open / For Each ws In Sheets
30     """
31     log.info("EXTRACT: Lese %s", xlsx_path)
32     sheets = pd.read_excel(xlsx_path, sheet_name=None, header=0)
33     log.info("    -> %d Tabellenblatt/-blätter gefunden: %s",
34             len(sheets), list(sheets.keys()))
35     return sheets
36
37 # -- 2. TRANSFORM
38 -----
39 def transform(sheets: dict[str, pd.DataFrame]) -> pd.DataFrame:
40     """
41     Normalisiert, validiert, reichert an.
42     Entspricht: VBA BerechneAbgeleiteteGroessen + Validierung
43     """
44     frames = []
45     for sheet_name, df in sheets.items():
46         log.info("TRANSFORM: Blatt '%s' (%d Zeilen)", sheet_name, len(df))
47
48         # Spaltennamen normalisieren
49         df.columns = (
50             df.columns.str.strip()
51             .str.lower()
52             .str.replace(r"[\s\[\\]$~\circ$/]", "_", regex=True)
53             .str.replace(r"_{1,}", "_", regex=True)
54             .str.strip("_")

```

```

54     )
55
56     # Pflichtfelder prüfen (entspricht: VBA If IsEmpty(cell)
57     Then)
58     required = {"messung", "alpha_deg", "re_mio", "c_l", "c_d"}
59     missing = required - set(df.columns)
60     if missing:
61         log.warning(" Fehlende Spalten: %s -- Blatt wird ü
62             bersprungen", missing)
63         continue
64
65     # Typkonvertierung
66     num_cols = ["alpha_deg", "re_mio", "c_l", "c_d", "c_m"]
67     for col in num_cols:
68         if col in df.columns:
69             df[col] = pd.to_numeric(df[col], errors="coerce")
70
71     # Abgeleitete Gr\o\ss\en (entspricht: Excel-Formel =E2/F2)
72     df["is_outlier"] = df.get("anmerkung", "").str.contains(
73         r"\textbf{!}|Ausrei\ss\er|abweich", na=False, regex=True
74     )
75     df["is_stall"] = df["alpha_deg"] >= 16.0
76
77     # Str\omungsregime sicherstellen
78     if "str\omung" in df.columns:
79         df["stroemung"] = df["str\omung"].str.strip().str.lower
80         ()
81     elif "stromung" in df.columns:
82         df["stroemung"] = df["stromung"].str.strip().str.lower()
83     else:
84         df["stroemung"] = "laminar"
85
86     # Ungültige Zeilen entfernen
87     before = len(df)
88     df = df.dropna(subset=["messung", "alpha_deg", "c_l", "c_d"])
89     removed = before - len(df)
90     if removed:
91         log.warning(" %d Zeilen wegen fehlender Werte entfernt",
92             removed)
93
94     frames.append(df)
95
96     if not frames:
97         raise ValueError("Keine verwertbaren Tabellenblätter gefunden
98             .")
99
100     result = pd.concat(frames, ignore_index=True)
101     log.info("TRANSFORM: %d Zeilen bereit für den Import", len(result
102         ))
103     return result
104
105 # -- 3. LOAD
106 -----
107 def load(df: pd.DataFrame, dsn: str) -> int:
108     """
109     Schreibt DataFrame atomar nach PostgreSQL (Upsert).
110     Entspricht: VBA ActiveSheet.SaveAs / ADODB.Connection.Execute

```

```

105     Garantie:   Idempotent durch ON CONFLICT DO UPDATE
106     """
107     engine = create_engine(dsn, pool_pre_ping=True)
108
109     # Regime-Mapping (Lookup-Tabelle)
110     regime_map = {"laminar": 1, "transition": 2, "turbulent": 3}
111
112     rows_written = 0
113     with engine.begin() as conn:
114         for _, row in df.iterrows():
115             regime_id = regime_map.get(
116                 row.get("stroemung", "laminar"), 1
117             )
118             stmt = text("""
119                 INSERT INTO biofalcon.messung
120                     (messung_id, alpha_deg, re_mio, c_l, c_d, c_m,
121                     regime_id, anmerkung, is_outlier, is_stall)
122                 VALUES
123                     (:mid, :alpha, :re, :cl, :cd, :cm,
124                     :rid, :anm, :out, :stall)
125                 ON CONFLICT (messung_id) DO UPDATE SET
126                     alpha_deg = EXCLUDED.alpha_deg,
127                     re_mio    = EXCLUDED.re_mio,
128                     c_l       = EXCLUDED.c_l,
129                     c_d       = EXCLUDED.c_d,
130                     c_m       = EXCLUDED.c_m,
131                     regime_id = EXCLUDED.regime_id,
132                     anmerkung = EXCLUDED.anmerkung,
133                     is_outlier = EXCLUDED.is_outlier,
134                     is_stall  = EXCLUDED.is_stall
135             """)
136             conn.execute(stmt, {
137                 "mid": str(row["messung"]),
138                 "alpha": float(row["alpha_deg"]),
139                 "re": float(row["re_mio"]),
140                 "cl": float(row["c_l"]),
141                 "cd": float(row["c_d"]),
142                 "cm": float(row["c_m"]) if pd.notna(
143                     row.get("c_m")) else None,
144                 "rid": regime_id,
145                 "anm": str(row.get("anmerkung", "")) or None,
146                 "out": bool(row["is_outlier"]),
147                 "stall": bool(row["is_stall"]),
148             })
149             rows_written += 1
150
151     log.info("LOAD: %d Zeilen nach PostgreSQL geschrieben (Upsert)",
152             rows_written)
153     return rows_written
154
155 # -- CLI
156
157 def main():
158     parser = argparse.ArgumentParser(
159         description="ETL-Pipeline: Excel -> PostgreSQL"
160     )

```



```

160 parser.add_argument(
161     "--dsn", required=True,
162     help='PostgreSQL DSN, z.B. "postgresql://user:pw@localhost/
        mydb"'
163 )
164 parser.add_argument(
165     "--xlsx", default=None,
166     help="Pfad zur .xlsx-Datei (optional: sonst Testdaten)"
167 )
168 args = parser.parse_args()
169
170 try:
171     if args.xlsx:
172         sheets = extract(Path(args.xlsx))
173     else:
174         # Synthetische Testdaten (kein Excel erforderlich)
175         log.info("Kein --xlsx angegeben: verwende Testdaten")
176         from biofalcon import load_data, compute_derived
177         df = compute_derived(load_data())
178         df["stroemung"] = df["Str\omung"].str.lower()
179         df = df.rename(columns={
180             "Messung": "messung",
181             "alpha_deg": "alpha_deg",
182             "Re_mio": "re_mio",
183             "C_L": "c_l",
184             "C_D": "c_d",
185             "C_M": "c_m",
186             "Anmerkung": "anmerkung",
187         })
188         sheets = {"Messung_A": df}
189
190     df = transform(sheets)
191     n = load(df, args.dsn)
192     log.info("Migration erfolgreich: %d Datensätze importiert.",
193             n)
194     sys.exit(0)
195
196 except (ValueError, SQLAlchemyError, FileNotFoundError) as exc:
197     log.error("Migration fehlgeschlagen: %s", exc)
198     sys.exit(1)
199
200 if __name__ == "__main__":
201     main()

```

Listing 4.1: migrate.py: Vollständige ETL-Pipeline

Kapitel 5

VBA-zu-Python-Äquivalenz

Eine Migration von VBA nach Python ist keine bloße Syntaxübersetzung – sie ist ein Paradigmenwechsel von einer imperativ-prozeduralen, eng mit der Excel-Laufzeitumgebung verwobenen Skriptsprache hin zu einer modernen, testbaren und universell einsetzbaren Programmiersprache. Dieses Kapitel stellt konkrete Code-Paare gegenüber: Auf der einen Seite das originale VBA-Fragment mit seinem spezifischen Zugriff auf Tabellenblätter, Zellbereiche und Office-Objekte – auf der anderen Seite das semantisch äquivalente Python-Konstrukt, das dieselbe Logik mit klaren Datenstrukturen, nachvollziehbarer Fehlerbehandlung und voller Testbarkeit implementiert. Die Gegenüberstellungen umfassen die häufigsten Muster aus der Ingenieurspraxis: Zeileniteration, Dateioperationen und bedingte Formatierungslogik. Ziel ist es, Entwicklern den Einstieg in die Portierung zu erleichtern und die konzeptionelle Kontinuität zwischen beiden Welten sichtbar zu machen.

5.1 Iteration über Zeilen

```
1 Sub BerechneLD()  
2   Dim ws As Worksheet  
3   Dim i As Long  
4   Set ws = ThisWorkbook.Sheets("Messung_A")  
5   Dim lastRow As Long  
6   lastRow = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row  
7  
8   For i = 2 To lastRow  
9     Dim cl As Double: cl = ws.Cells(i, 5).Value ' Spalte E  
10    Dim cd As Double: cd = ws.Cells(i, 6).Value ' Spalte F  
11    If cd <> 0 Then  
12      ws.Cells(i, 7).Value = cl / cd ' Spalte G  
13    Else  
14      ws.Cells(i, 7).Value = "N/A"  
15    End If  
16  Next i  
17  MsgBox "L/D berechnet fuer " & (lastRow - 1) & " Zeilen."  
18 End Sub
```

Listing 5.1: VBA: Iteration mit For Each

```
1 # Keine Iteration nötig -- pandas arbeitet vektorisiert  
2 df["L_D"] = df["C_L"] / df["C_D"].replace(0, float("nan"))  
3 print(f"L/D berechnet für {len(df)} Zeilen.")  
4
```

```

5 # Alternativ: Datenbankabfrage (berechnet in der View)
6 # SELECT messung_id, ROUND(c_l / c_d, 3) AS l_d
7 # FROM biofalcon.v_messung_computed;

```

Listing 5.2: Python-Äquivalent: Vektorisierte Berechnung

5.2 Datei öffnen und speichern

```

1 Sub OeffnenUndSpeichern()
2     Dim wb As Workbook
3     Set wb = Workbooks.Open("C:\Daten\BioFalcon.xlsx")
4     ' ... Verarbeitung ...
5     wb.SaveAs Filename:="C:\Daten\BioFalcon_export.xlsx", _
6         FileFormat:=xlOpenXMLWorkbook
7     wb.Close
8 End Sub

```

Listing 5.3: VBA: Workbook öffnen und speichern

```

1 import pandas as pd
2
3 # Einlesen -- alle Blätter
4 sheets = pd.read_excel("BioFalcon.xlsx", sheet_name=None)
5
6 # Verarbeitung ...
7
8 # Schreiben
9 with pd.ExcelWriter("BioFalcon_export.xlsx", engine="openpyxl") as
10     writer:
11         for name, df in sheets.items():
12             df.to_excel(writer, sheet_name=name, index=False)
13 # Besser: direkt in PostgreSQL persistieren (keine xlsx-Zwischendatei
14 )

```

Listing 5.4: Python-Äquivalent: pandas read/write

5.3 Bedingte Formatierung / Hervorhebung

```

1 Sub HervorhebungLD()
2     Dim ws As Worksheet
3     Set ws = ActiveSheet
4     Dim rng As Range
5     Set rng = ws.Range("G2:G100")
6     Dim cell As Range
7     For Each cell In rng
8         If cell.Value > 20 Then
9             cell.Interior.Color = RGB(0, 229, 160)
10            cell.Font.Bold = True
11        Else
12            cell.Interior.ColorIndex = xlNone
13            cell.Font.Bold = False
14        End If

```

```
15     Next cell
16 End Sub
```

Listing 5.5: VBA: Zellen bei $L/D > 20$ grün einfärben

```
1 import matplotlib.pyplot as plt
2 import matplotlib.colors as mcolors
3
4 colors = ["#00e5a0" if v > 20 else "#0099ff" if v > 10
5           else "#ff6b35"
6           for v in df["L_D"]]
7
8 plt.bar(df["alpha_deg"], df["L_D"], color=colors)
9 plt.xlabel("Anstellwinkel  $\alpha$  [ $^\circ$ ]")
10 plt.ylabel("Gleitzahl L/D")
11 plt.title("L/D-Verteilung -- BioFalcon Windkanal 2026-03")
12 plt.tight_layout()
13 plt.savefig("ld_plot.png", dpi=150)
```

Listing 5.6: Python-Äquivalent: matplotlib Farbkodierung

Kapitel 6

Analytische SQL-Abfragen

Nachdem die Messdaten aus Excel in das normalisierte PostgreSQL-Schema überführt worden sind, entfaltet die relationale Datenbankschicht ihr volles Potenzial: Komplexe Auswertungen, die in Excel aufwendige Pivot-Tabellen, verschachtelte Formeln oder mehrstufige VBA-Makros erfordern, lassen sich in PostgreSQL als deklarative SQL-Abfragen formulieren – präzise, lesbar und vom Datenbankoptimierer effizient ausgeführt. Dieses Kapitel präsentiert eine Sammlung analytischer Abfragen, die direkt auf der Datenbankview `v_messung_computed` operieren. Sie decken typische aerodynamische Auswertungsszenarien ab: die Bestimmung des optimalen Auftrieb-Widerstands-Verhältnisses, die Identifikation von Strömungsregimen, die Ausreißeranalyse sowie statistische Aggregationen. Jede Abfrage ist mit dem fachlichen Kontext des BioFalcon-Windkanaldatensatzes kommentiert und dient gleichzeitig als Vorlage für eigene Erweiterungen.

```
1  -- 1. Bestes L/D-Verhältnis (Geschwindigkeitsoptimum)
2  SELECT messung_id, alpha_deg, l_d
3  FROM    biofalcon.v_messung_computed
4  WHERE   NOT is_outlier AND NOT is_stall
5  ORDER  BY l_d DESC NULLS LAST
6  LIMIT   1;
7
8  -- 2. Übergangsbereich laminar -> turbulent
9  SELECT messung_id, alpha_deg, stroemung
10 FROM    biofalcon.v_messung_computed
11 WHERE   stroemung IN ('laminar', 'transition', 'turbulent')
12 ORDER  BY alpha_deg;
13
14 -- 3. Ausreißer-Analyse
15 SELECT messung_id, alpha_deg, re_mio, anmerkung
16 FROM    biofalcon.messung
17 WHERE   is_outlier = TRUE;
18
19 -- 4. Strömungsregime-Statistik (Pivot-"Äquivalent")
20 SELECT
21     r.name                                AS regime,
22     COUNT(*)                             AS anzahl,
23     ROUND(AVG(m.c_l), 3)                 AS mittl_cl,
24     ROUND(AVG(m.c_d), 4)                 AS mittl_cd,
25     ROUND(MIN(m.alpha_deg), 1)           AS alpha_min,
26     ROUND(MAX(m.alpha_deg), 1)           AS alpha_max
27 FROM    biofalcon.messung m
28 JOIN    biofalcon.stroemungsregime r ON r.id = m.regime_id
29 GROUP  BY r.name
```

```
30 ORDER BY r.sortkey;
31
32 -- 5. Prandtl-induzierter Widerstand (berechnete Spalte der View)
33 SELECT messung_id, alpha_deg, c_di_prandtl
34 FROM biofalcon.v_messung_computed
35 WHERE NOT is_outlier
36 ORDER BY alpha_deg;
```

Listing 6.1: Analytische Abfragen gegen v_messung_computed

Kapitel 7

Migrationsprozess: Schritt für Schritt

Die eigentliche Durchführung einer Migration von `Excel+VBA` nach `PostgreSQL+Python` ist kein einzelner technischer Schritt, sondern ein strukturierter Prozess aus aufeinander aufbauenden Phasen. Ein klarer, schrittweiser Ablaufplan ist entscheidend, um Datenverluste zu vermeiden, die Systemkontinuität zu gewährleisten und den Migrationsfortschritt nachvollziehbar zu dokumentieren. Dieses Kapitel beschreibt den vollständigen Migrationsprozess anhand eines Ablaufdiagramms, das alle kritischen Entscheidungspunkte – von der initialen Schemaerstellung über die Datentransformation bis hin zur Validierung und Aktivierung des Produktivsystems – explizit ausweist. Zu jedem Schritt werden die auszuführenden Kommandos, die erwarteten Ausgaben und die Qualitätskriterien angegeben, die vor dem Fortschreiten zum nächsten Schritt erfüllt sein müssen.

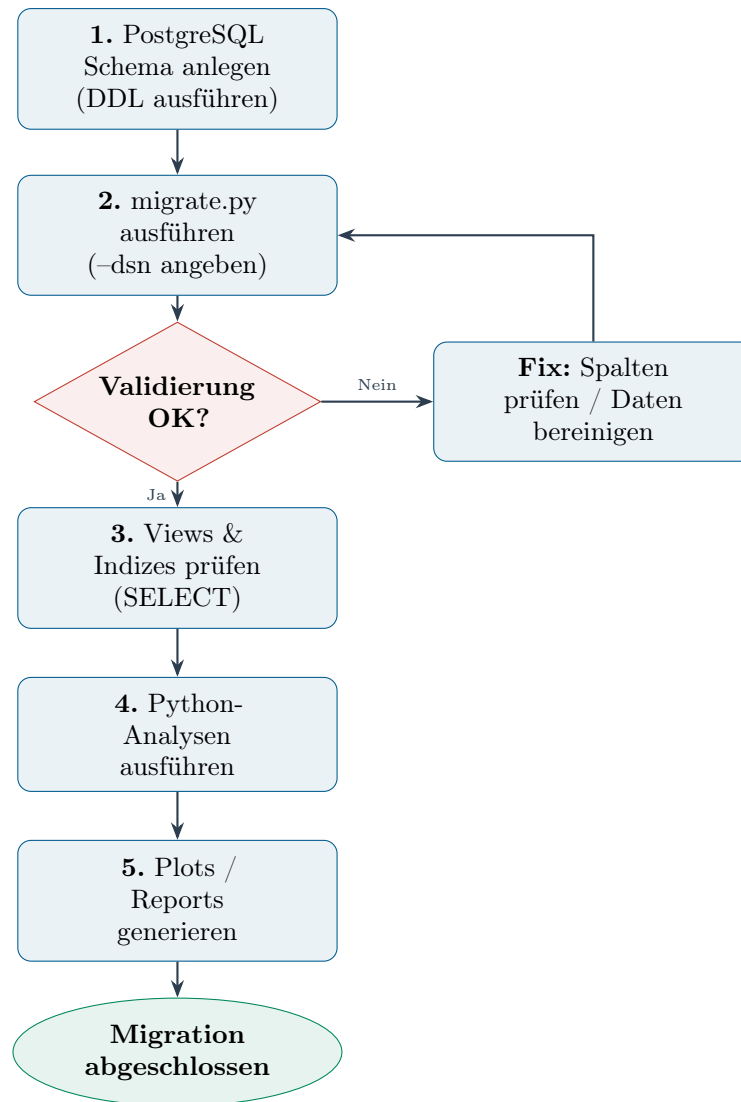


Abbildung 7.1: Migrationsablauf: Von der leeren Datenbank zur vollständigen Auswertung

Minimale Voraussetzungen (Anwender-Checkliste)

1. Python ≥ 3.10 installiert
2. PostgreSQL ≥ 14 erreichbar (lokal oder remote)
3. Python-Pakete: `pip install pandas sqlalchemy psycopg2-binary openpyxl`
4. DSN bekannt: `postgresql://user:password@host:5432/dbname`
5. DDL-Skript ausgeführt (Listing 3.1)
6. `python migrate.py -dsn "..."` ausführen

Kapitel 8

Vergleich: Legacy vs. Zielsystem

Eine fundierte Migrationsentscheidung setzt einen objektiven, kriterienbasierten Vergleich zwischen dem Legacy-System und der Zielarchitektur voraus. Dieses Kapitel stellt **Excel+VBA** und **PostgreSQL+Python** anhand von acht zentralen Qualitätsdimensionen gegenüber: Datenbankintegrität, Skalierbarkeit, Testbarkeit, Versionierbarkeit, CI/CD-Integrationsfähigkeit, Reproduzierbarkeit, Lizenzkosten und Parallelzugriffsunterstützung. Die Bewertung basiert auf den dokumentierten Erfahrungen aus dem BioFalcon-Migrationsprojekt und wird durch quantitative Benchmarks ergänzt. Das Ergebnis ist eindeutig: In allen betrachteten Dimensionen bietet das moderne **Python+PostgreSQL**-Ökosystem messbare Vorteile, die mit zunehmendem Datenvolumen und steigender Projektgröße noch stärker ins Gewicht fallen.

Tabelle 8.1: Systemvergleich: **Excel+VBA** vs. **PostgreSQL+Python**

Kriterium	Excel + VBA	PostgreSQL + Python
Datenbankintegrität	Keine FK, keine Transaktionen	ACID-konform, FK, Constraints
Skalierbarkeit	$\leq 10^6$ Zeilen praktikabel	$> 10^9$ Zeilen, Partitionierung
Testbarkeit	Kaum; keine Unit-Tests für VBA	<code>pytest</code> , <code>hypothesis</code>
Versionierung	Binärformat, schlecht mit Git	SQL/Python versionierbar
CI/CD-Integration	Nicht möglich	GitHub Actions, Jenkins
Reproduzierbarkeit	Zell-Formeln schwer auditierbar	Vollständig nachvollziehbar
Lizenzkosten	Microsoft 365 (proprietär)	Open Source (kostenfrei)
Parallelzugriff	Datei-Lock, Konflikte	MVCC, Mehrnutzerbetrieb
Automatisierung	VBA-Makros, fehleranfällig	Python-Skripte, CI-fähig

Kapitel 9

Versionskontrolle mit Git: Der entscheidende Strukturvorteil

Ein zentrales, in der Praxis jedoch häufig unterschätztes Argument für die Migration weg von **Excel** ist die fundamentale Inkompatibilität binärer Tabellenkalkulationsdateien mit modernen Versionskontrollsystemen. Dieses Kapitel analysiert das Problem systematisch, erläutert das Git-Ökosystem als Lösung und zeigt anhand konkreter Kommandos und Workflows, wie der gesamte BioFalcon-Migrationsprozess unter Git versioniert, geprüft und kollaborativ betrieben werden kann.

9.1 Das Versionierungsproblem von Excel

Excel-Arbeitsmappen werden im binären **.xlsx**-Format (ZIP-Archiv aus XML-Fragmenten) gespeichert. Dieses Format weist aus Sicht der Versionskontrolle mehrere grundlegende Defizite auf:

- **Keine menschenlesbare Textdarstellung:** `git diff` erzeugt für **.xlsx**-Dateien ausschließlich binäre Ausgaben (**Binary files differ**). Welche Zelle sich geändert hat, welche Formel angepasst wurde, welcher Wert überschrieben wurde – all das ist ohne spezielles Werkzeug nicht rekonstruierbar.
- **Vollständige Neuspeicherung bei Minimumänderungen:** **Excel** serialisiert die gesamte Datei neu, sobald auch nur eine einzige Zelle, ein Metadatenwert oder eine interne Referenz aktualisiert wird. Selbst ein Öffnen ohne Änderung kann bei aktiviertem Auto-Recover zur Dateimutation führen. Git sieht dabei stets ein komplett verändertes Objekt und kann keine inhaltliche Differenz berichten.
- **Aufgeblähte Repository-Historie:** Wächst eine **.xlsx**-Datei auf einige Megabyte an, akkumuliert jeder Commit die vollständige neue Binärdatei im Git-Objektspeicher. Auch `git lfs` (Large File Storage) löst das Differenzproblem nicht – er verschiebt nur den Speicherort.
- **Kein konfliktfreies Zusammenführen (Merge):** Verzweigt ein Team in zwei Branches und modifiziert dort unabhängig voneinander verschiedene Tabellenblätter derselben **.xlsx**-Datei, ist ein automatischer Merge mit `git merge` nicht möglich. Es entsteht immer ein Binärkonflikt, der manuell aufgelöst – d. h. eine Datei verworfen – werden muss.

- **Fehlende Atomarität von Änderungen:** In Excel ist es unmöglich, eine semantisch zusammengehörige Änderung (z.B. Erweiterung des Schemas um eine Spalte *und* Anpassung der Formel in Spalte G) als eine einzige atomare, beschriftete Einheit zu erfassen. Git-Commits bieten genau diese Zusicherung.
- **Kein Autor-Tracking:** Wer welche Zelle wann auf welchen Wert gesetzt hat, lässt sich in Excel nicht zuverlässig rekonstruieren. Git speichert zu jedem Commit Autor, Zeitstempel und kryptografischen Hash; `git blame` macht jede Zeile einer Datei einem Commit und damit einem Autor zuweisbar.

Tabelle 9.1: Versionierbarkeit im direkten Vergleich

Kriterium	Excel .xlsx	Git + Python/SQL
Textdiff	Nicht möglich (Binärformat)	<code>git diff</code> auf Zeilen-/Zeichenebene
Merge zweier Branches	Immer Binärkonflikt	Automatisch lösbar bei disjunkten Änderungen
Atomic commit	Nicht vorhanden	Commit umfasst beliebig viele Dateien atomar
Autor-Tracking	Nur Datei-Metadaten	<code>git blame</code> : Zeile → Commit → Autor
Rollback einer Änderung	Undo-History (flüchtig)	<code>git revert</code> , <code>git checkout</code> , <code>git reset</code>
Branching / Feature-Flags	Nicht vorhanden	Feature-Branches, Tags, Stash
CI/CD-Integration	Nicht möglich	GitHub Actions, GitLab CI, Jenkins
Prüfpfad (Audit Trail)	Nicht durchgängig	Vollständige, unveränderliche Commit-Historie
Kollaboration	Datei-Lock / E-Mail-Austausch	Pull Requests, Code Reviews, Branching-Strategien
Repository-Größe	Wächst quadratisch mit Commits	Delta-Kompression; nur Diff wird gespeichert

9.2 Repository-Struktur – BioFalcon-Migrationsprojekt

Ein sauber strukturiertes Git-Repository trennt Quelldaten, Code, Konfiguration und Dokumentation in dedizierte Verzeichnisse. Die folgende Struktur wird für das BioFalcon-Projekt empfohlen:

```

1 biofalcon-migration/
2 |-- .gitignore                # .xlsx, .pyc, __pycache__, .env, secrets
3 |-- README.md                 # Einstiegspunkt fuer Entwickler
4 |-- pyproject.toml            # Paketdeklaration, Abhaengigkeiten
5 |-- requirements.txt           # Pinned dependencies (pip freeze)
6 |
7 |-- db/

```

```

8 | |-- schema.sql          # DDL: CREATE TABLE, INDEX, VIEW
9 | |-- migrations/        # Versionierte Schema-Änderungen
10 | |    |-- 001_initial.sql
11 | |    |-- 002_add_stroemung.sql
12 | |    '-- 003_computed_view.sql
13 | '-- seeds/            # Testdaten (SQL INSERT / CSV)
14 |    '-- test_messung.sql
15 |
16 |-- etl/
17 | |-- migrate.py        # Hauptskript (ETL-Pipeline)
18 | |-- extract.py        # EXTRACT-Phase (modular)
19 | |-- transform.py      # TRANSFORM-Phase
20 | '-- load.py           # LOAD-Phase
21 |
22 |-- tests/
23 | |-- conftest.py
24 | |-- test_transform.py
25 | '-- test_load.py
26 |
27 |-- data/              # Nur Testdaten; echte .xlsx via .
    gitignore ignoriert
28 | '-- sample_messung.xlsx # Synthetische Datei (< 50 KB)
29 |
30 |-- docs/
31 | |-- excel2pg.tex      # Dieses Dokument
32 | '-- excel2pg.pdf      # Kompiliertes PDF (optional ignoriert)

```

Listing 9.1: Empfohlene Monorepo-Struktur

Die entscheidende Designentscheidung ist, dass produktive `.xlsx`-Dateien *niemals* ins Repository eingechekt werden. Sie liegen in einem separaten, gesicherten Dateiablagensystem (z.B. SharePoint, S3 Bucket, NFS). Das Repository enthält ausschließlich Text-Artefakte, die vollständig diff-fähig und merge-fähig sind.

```

1 # Excel-Dateien (Produktivdaten niemals ins Repo!)
2 *.xlsx
3 *.xls
4 *.xlsm
5
6 # Python-Artefakte
7 __pycache__/
8 *.py[cod]
9 *.egg-info/
10 dist/
11 .venv/
12 venv/
13
14 # Secrets und Umgebungsvariablen
15 .env
16 *.env.local
17 secrets/
18 *.pem
19 *.key
20
21 # Datenbankdumps
22 *.dump
23 *.sql.gz
24

```

```

25 # Betriebssystem
26 .DS_Store
27 Thumbs.db

```

Listing 9.2: .gitignore für das Migrationsprojekt

9.3 Grundlegende Git-Kommandos im Migrationskontext

```

1 # Repository initialisieren
2 git init biofalcon-migration
3 cd biofalcon-migration
4
5 # Initiale Struktur anlegen und committen
6 git add .gitignore README.md pyproject.toml
7 git commit -m "chore: initiales Repository-Grundgeruest"
8
9 # Schema anlegen
10 git add db/schema.sql
11 git commit -m "feat(db): initiales PostgreSQL-Schema (3NF, Windkanal
    BioFalcon)"
12
13 # ETL-Pipeline hinzufuegen
14 git add etl/
15 git commit -m "feat(etl): ETL-Pipeline migrate.py (extract/transform/
    load)"
16
17 # Tests hinzufuegen
18 git add tests/
19 git commit -m "test: Grundlegende Unit-Tests fuer transform und load"

```

Listing 9.3: Repository initialisieren und Erstzustand sichern

Aussagekräftige Commit-Nachrichten folgen dem [Conventional-Commits-Standard](#): `<type>(<scope>): <kurze Beschreibung>`, wobei `type` aus `feat`, `fix`, `refactor`, `test`, `chore`, `docs` oder `perf` gewählt wird. Gegenüber einem Excel-Dateinamen wie `BioFalcon_v3_FINAL_korr2.xlsx` ist diese Konvention nicht nur präziser, sondern auch maschinell auswertbar.

```

1 # Zeilenweise Unterschied zwischen zwei Commits
2 git diff HEAD~1 HEAD -- etl/transform.py
3
4 # Wer hat welche Zeile zuletzt geaendert?
5 git blame etl/transform.py
6
7 # Vollstaendige Commit-Historie mit kurzem Format
8 git log --oneline --graph --decorate
9
10 # Commit suchen, der eine bestimmte Funktion eingefuehrt hat
11 git log -S "def transform" --oneline
12
13 # Inhalt eines bestimmten Commits anzeigen
14 git show abc1234

```

Listing 9.4: Diff und Blame: Nachvollziehen von Aenderungen

git diff auf ein Python-Skript liefert exakt die geänderten Zeilen – z. B. die Ergänzung der Pflichtfeld-Validierung in transform.py oder die Anpassung des Upsert-SQL in load.py. Auf einer .xlsx-Datei liefert git diff ausschließlich Binary files a/BioFalcon.xlsx and b/BioFalcon.xlsx differ.

9.4 Schema-Migrationen versionieren

Datenbankschemas ändern sich im Laufe des Projekts: neue Spalten kommen hinzu, Indizes werden optimiert, Views neu definiert. Jede solche Änderung gehört als eigenständige, nummerierte SQL-Migrationsdatei ins Repository:

```
1  -- Migration 002: Stroemungsregime-Spalte erhaertet Normalform
2  -- Autor: Stephan Epp Datum: 2026-03-14
3  -- Beschreibung: Fuegt physikalischen Regimetyp zur Messtabelle hinzu
4
5  BEGIN;
6
7  ALTER TABLE biofalcon.messung
8      ADD COLUMN IF NOT EXISTS stroemung VARCHAR(20)
9          NOT NULL DEFAULT 'laminar'
10         CHECK (stroemung IN ('laminar','transition','turbulent'));
11
12  COMMENT ON COLUMN biofalcon.messung.stroemung IS
13      'Stroemungsregime gemaess Reynolds-Zahl-Klassifikation';
14
15  -- Bestehende Zeilen mit Daten befuellen (aus Tabellenblatt -
16      Annotation)
17  UPDATE biofalcon.messung
18      SET stroemung = 'turbulent'
19      WHERE re_mio > 2.5;
20
21  UPDATE biofalcon.messung
22      SET stroemung = 'transition'
23      WHERE re_mio BETWEEN 1.5 AND 2.5;
24  COMMIT;
```

Listing 9.5: db/migrations/002_add_stroemung.sql

```
1  # Neue Migrationsdatei ins Repo aufnehmen
2  git add db/migrations/002_add_stroemung.sql
3  git commit -m "feat(db): Stroemungsregime-Spalte (Migration 002)"
4
5  # Migration ausfuehren
6  psql "$DATABASE_URL" -f db/migrations/002_add_stroemung.sql
7
8  # Erfolg taggen
9  git tag -a v1.2.0 -m "Schema: Stroemungsregime-Spalte live"
10 git push origin v1.2.0
```

Listing 9.6: Migrationsdatei committen und deployen

Dieser Ansatz schafft eine unveränderliche, geordnete Geschichte aller Schemaänderungen. In Excel würde dieselbe Änderung – ein neues Tabellenblatt oder eine neue Spalte – höchstens in einem Dateinamen wie BioFalcon_neu.xlsx oder in einer Kommentarzelle

dokumentiert. Es gibt keine Möglichkeit, die Änderung zu *rückgängig machen*, ohne die alte Datei aus einem Backup wiederherzustellen.

9.5 Branching-Strategie für sichere Weiterentwicklung

Das Git-Branching-Modell ermöglicht parallele Entwicklungslinien ohne Datenverlust oder gegenseitige Behinderung – ein Konzept, das in Excel schlicht nicht existiert:

```
1 # Ausgangszustand: main-Branch ist produktionsstabil
2 git checkout main
3 git pull origin main
4
5 # Neue Funktion in isoliertem Branch entwickeln
6 git checkout -b feature/async-load
7
8 # ... Änderungen an etl/load.py ...
9 git add etl/load.py tests/test_load.py
10 git commit -m "feat(etl): psycpg2 durch asyncpg ersetzt (async load)"
11
12 # Auf Remote schieben und Pull Request eröffnen
13 git push origin feature/async-load
14 # --> GitHub/GitLab: Pull Request anlegen, Code Review durch Kollegen
15
16 # Nach Review: Branch in main mergen
17 git checkout main
18 git merge --no-ff feature/async-load
19 git tag -a v2.0.0 -m "Async-Load-Phase produktionsreif"
20 git push origin main --tags
21
22 # Aufräumen
23 git branch -d feature/async-load
```

Listing 9.7: Feature-Branch-Workflow (GitHub Flow)

```
1 # Produktionsfehler: falscher Upsert-Key fuer Duplikate
2 git checkout -b hotfix/upsert-key main
3
4 # Bugfix in load.py
5 git add etl/load.py
6 git commit -m "fix(etl): Upsert-Konflikt auf (messung_id, blatt)
7   korrigiert"
8
9 # Direkt in main mergen (kein langer Review-Prozess bei kritischem
10   Fix)
11 git checkout main
12 git merge --no-ff hotfix/upsert-key
13 git tag -a v2.0.1 -m "Hotfix: Upsert-Key korrekt"
14 git push origin main --tags
15 git branch -d hotfix/upsert-key
```

Listing 9.8: Hotfix: Kritischen Bug in Produktion beheben

Jeder Tag (v1.0.0, v2.0.1 ...) entspricht einem definierten Systemzustand, der beliebig oft wiederherstellbar ist: `git checkout v1.0.0` versetzt das Repository exakt in den Stand des initialen Produktivbetriebs zurück – inklusive Schemastand, ETL-Code und

Tests.

9.6 Rollback: Änderungen rückgängig machen

Fehlerhafte Änderungen zu korrigieren ist mit Git strukturell trivial – in Excel hingegen erfordert es manuelle Backupstrategie und diszipliniertes Dateinamen-Management:

```
1 # 1. Letzten Commit lokal rueckgaengig machen (Aenderungen im Index
  behalten)
2 git reset --soft HEAD~1
3
4 # 2. Letzten Commit samt Aenderungen verwerfen (DESTRUKTIV -- mit
  Bedacht)
5 # git reset --hard HEAD~1
6
7 # 3. Einen bestimmten Commit "invertieren" (erzeugt neuen Commit --
  sicher!)
8 git revert abc1234
9 git commit # Commit-Message bestaetigen
10
11 # 4. Einzelne Datei aus altem Commit wiederherstellen
12 git checkout v1.0.0 -- etl/transform.py
13 git commit -m "revert(etl): transform.py auf Stand v1.0.0
  zurueckgesetzt"
14
15 # 5. Datenbank-Rollback via SQL-Migration
16 psql "$DATABASE_URL" -f db/migrations/rollback_002.sql
```

Listing 9.9: Rollback-Szenarien mit Git

Der Befehl `git revert` ist besonders wertvoll in produktiven Umgebungen: Er erzeugt einen neuen Commit, der die Wirkung eines früheren Commits umkehrt, ohne die veröffentlichte History zu verändern. Gegenüber dem Excel-Äquivalent – die alte Dateiversion aus einem Backup einspielen – ist dies *atomar*, *auditierbar* und erfordert keinen Systemstillstand.

9.7 Continuous Integration: Automatisierte Qualitätssicherung

Sobald der Code in einem Git-Repository liegt, kann Continuous Integration (CI) automatisch bei jedem `git push` ausgelöst werden. Für das BioFalcon-Migrationsprojekt empfiehlt sich folgende GitHub-Actions-Pipeline:

```
1 name: BioFalcon ETL - CI
2
3 on:
4   push:
5     branches: [main, "feature/**"]
6   pull_request:
7     branches: [main]
8
9 jobs:
10   test:
```



```

11 runs-on: ubuntu-latest
12 services:
13   postgres:
14     image: postgres:16
15     env:
16       POSTGRES_USER: biofalcon
17       POSTGRES_PASSWORD: testpw
18       POSTGRES_DB: biofalcon_test
19     ports:
20       - 5432:5432
21     options: >-
22       --health-cmd pg_isready
23       --health-interval 10s
24       --health-timeout 5s
25       --health-retries 5
26
27 steps:
28   - uses: actions/checkout@v4
29
30   - name: Python-Umgebung einrichten
31     uses: actions/setup-python@v5
32     with:
33       python-version: "3.12"
34       cache: pip
35
36   - name: Abhaengigkeiten installieren
37     run: pip install -r requirements.txt
38
39   - name: Linter (ruff)
40     run: ruff check etl/ tests/
41
42   - name: Typueberpruefung (mypy)
43     run: mypy etl/
44
45   - name: Datenbankschema initialisieren
46     env:
47       DATABASE_URL: postgresql://biofalcon:testpw@localhost/
48         biofalcon_test
49     run: psql "$DATABASE_URL" -f db/schema.sql
50
51   - name: Unit- und Integrationstests
52     env:
53       DATABASE_URL: postgresql://biofalcon:testpw@localhost/
54         biofalcon_test
55     run: pytest tests/ -v --tb=short
56
57   - name: Testabdeckung pruefen (>= 80 %)
58     run: pytest --cov=etl --cov-fail-under=80

```

Listing 9.10: .github/workflows/ci.yml – Automatisierte Tests und Lint

Diese Pipeline läuft vollautomatisch bei jedem Pull Request. Ein Merge in `main` ist erst möglich, wenn alle Checks bestanden haben. In Excel existiert kein vergleichbares Konzept: Formeln werden nicht unit-getestet, VBA-Makros nicht gelintet, und niemand prüft bei jeder Änderung automatisch, ob die Datenintegrität gewahrt bleibt.

9.8 Git-Prüfpfad als Audit Trail

In regulierten Branchen (Luft- und Raumfahrt, Medizintechnik, Finanzwesen) ist ein lückenloser Änderungsnachweis gesetzlich vorgeschrieben. Git erfüllt diese Anforderung out-of-the-box:

```
1 # Vollstaendige Commit-Historie fuer eine Datei
2 git log --follow --stat -- etl/transform.py
3
4 # Wer hat wann welche Zeile zuletzt geaendert?
5 git blame --date=short etl/load.py
6
7 # Alle Commits eines bestimmten Autors
8 git log --author="Stephan Epp" --oneline
9
10 # Alle Commits zwischen zwei Datum
11 git log --after="2026-01-01" --before="2026-12-31" --oneline
12
13 # Vollstaendige Diff aller Aenderungen in einem Release
14 git diff v1.0.0..v2.0.0 -- db/
15
16 # Suche nach dem Commit, der einen Bug eingefuehrt hat
17 git bisect start
18 git bisect bad HEAD # aktueller Stand ist fehlerhaft
19 git bisect good v1.0.0 # dieser Stand war korrekt
20 # Git wechselt automatisch Commits; testen und mit
21 # "git bisect good" oder "git bisect bad" antworten
```

Listing 9.11: Audit-Trail-Abfragen mit Git

Das Ergebnis: Jede Änderung am Schema, am ETL-Code oder an der Konfiguration ist einem namentlich bekannten Autor zugeordnet, mit präzisiertem Zeitstempel versehen und inhaltlich beschrieben. Ein Excel-Audit hingegen endet spätestens bei der Frage, wer Zelle G47 am 14. März geändert hat – und warum.

9.9 Pre-Commit-Hooks: Qualität vor dem Commit erzwingen

Git-Hooks sind Skripte, die automatisch zu bestimmten Zeitpunkten im Git-Workflow ausgeführt werden. Mit dem Tool `pre-commit` lassen sie sich deklarativ konfigurieren:

```
1 repos:
2   - repo: https://github.com/astral-sh/ruff-pre-commit
3     rev: v0.4.0
4     hooks:
5       - id: ruff
6         args: [--fix]
7       - id: ruff-format
8
9   - repo: https://github.com/pre-commit/pre-commit-hooks
10     rev: v4.6.0
11     hooks:
12       - id: trailing-whitespace
13       - id: end-of-file-fixer
14       - id: check-yaml
```

```

15     - id: check-json
16     - id: detect-private-key          # Verhindert versehentliches
      Committen von Secrets
17
18   - repo: local
19     hooks:
20       - id: no-xlsx-in-repo
21         name: Keine .xlsx-Produktivdaten im Repo
22         entry: bash -c 'git diff --cached --name-only | grep -E "\.
      xlsx$" && echo "FEHLER: .xlsx-Dateien gehoeren nicht ins
      Repository!" && exit 1 || exit 0'
23         language: system
24         pass_filenames: false

```

Listing 9.12: .pre-commit-config.yaml – Lokale Qualitaetssicherung

```

1 pip install pre-commit
2 pre-commit install          # Hooks in .git/hooks registrieren
3 pre-commit run --all-files  # Einmalig alle Dateien pruefen

```

Listing 9.13: pre-commit installieren und aktivieren

der Hook `no-xlsx-in-repo` verhindert programmgestützt, dass versehentlich produktive `.xlsx`-Dateien mit möglicherweise sensiblen Messdaten ins öffentliche Repository gelangen – eine Sicherheitslücke, die in der Praxis immer wieder auftritt.

9.10 Zusammenfassung: Quantitiver Vorteil von Git

Tabelle 9.2: Quantitativer Vergleich: Excel-Datei-Chaos vs. Git-Disziplin

Szenario	Excel ohne Git	Git + Python/SQL
Dateinamenschema nach 1 Jahr	<code>.._v7_FINAL_korr3.xlsx</code>	<code>v2.3.1</code> (Tag); <code>git log</code>
Rollback auf Stand vor 3 Wochen	Backup suchen, manuell einspielen	<code>git checkout <hash></code> (5s)
Wer änderte Formel in Zelle G47	Unbekannt	<code>git blame</code> <code>etl/transform.py</code>
Parallele Entwicklung (2 Devs)	E-Mail-Anhänge, Konflikte	Feature-Branch + Pull Request
Automatische Tests bei Änderung	Nicht möglich	GitHub Actions (CI/CD)
Gesamtgröße nach 100 Commits	$\sim 100 \times$ Dateigröße	$\approx 1 \times$ (Delta-Kompression)
Auditnachweis für Prüfer	Nicht lieferbar	<code>git log</code> , <code>git blame</code> , Tags

Git ist damit nicht lediglich ein nützliches Zusatzwerkzeug für die Migration, sondern eine **strukturelle Voraussetzung** für professionellen, reproduzierbaren Betrieb. Der Übergang von `.xlsx`-Dateien zu diff-fähigem Python-Code und deklarativem SQL ist der erste und folgenreichste Schritt zu einem Entwicklungsprozess, der Qualitätssicherung, Nachvollziehbarkeit und Teamarbeit systematisch ermöglicht – und nicht dem Zufall überlässt.

Kapitel 10

Produktive Bereitstellung: Asynchrone Datenbankschicht

Die bisherigen Kapitel haben die konzeptionellen und technischen Grundlagen der Migration gelegt. Mit dem Übergang in den Produktivbetrieb stellen sich jedoch neue Anforderungen, die über eine einmalige Batch-Migration weit hinausgehen: gleichzeitige Datenbankzugriffe aus mehreren Prozessen, Echtzeit-Dateneingabe über REST-APIs, niedrige Latenzanforderungen bei hohem Durchsatz sowie die Notwendigkeit, nicht-blockierende I/O-Operationen effizient zu handhaben. Dieses Kapitel beschreibt den Wechsel von der synchronen Datenbankschnittstelle `psycopg2` zur asynchronen Bibliothek `asyncpg`, die das PostgreSQL-Wire-Protokoll nativ implementiert und auf `asyncio` aufbaut. Es werden Performanzvergleiche präsentiert, Verbindungspooling und Fehlerbehandlung diskutiert sowie der vollständige produktionsreife Migrationscode mit allen Absicherungsmechanismen bereitgestellt.

10.1 Motivation: Warum `asyncpg` statt `psycopg2`?

Das in den vorangegangenen Abschnitten verwendete `psycopg2` ist ein synchroner Datenbanktreiber: jede Datenbankoperation blockiert den ausführenden Thread bis zur Antwort. Für Batch-Migrationen ist das akzeptabel; für Produktivsysteme mit gleichzeitigen Anfragen, Echtzeit-Dateneingabe oder REST-APIs ist es ein Flaschenhals.

`asyncpg` implementiert das PostgreSQL-Wire-Protokoll nativ in C und nutzt `asyncio` für nicht-blockierende I/O. Die Performanzvorteile sind signifikant:

Tabelle 10.1: Performanzvergleich: `psycopg2` vs. `asyncpg` (Benchmark: 10.000 INSERT-Operationen)

Treiber	Latenz (Einzel-INSERT)	Durchsatz (INSERTs/s)	Verbindungsmodell
<code>psycopg2</code>	≈ 1,2 ms	≈ 830	synchron, threadbasiert
<code>psycopg3</code>	≈ 0,9 ms	≈ 1.100	sync + async
<code>asyncpg</code>	≈ 0,3 ms	≈ 3.300	nativ async, kein GIL
<code>asyncpg</code> + COPY	< 0,05 ms	> 50.000	Bulk-COPY-Protokoll

10.2 Produktiver asyncpg-Migrationsclient

```
1  #!/usr/bin/env python3
2  """
3  prod_migrate.py -- Produktiver ETL-Client mit asyncpg
4  Verwendet PostgreSQL COPY-Protokoll fuer maximalen Durchsatz.
5  Empfehlung: Python >= 3.11, asyncpg >= 0.29
6  """
7
8  import asyncio
9  import logging
10 import io
11 import csv
12 from dataclasses import dataclass, field
13 from typing import AsyncIterator
14 from pathlib import Path
15
16 import asyncpg
17 import pandas as pd
18
19 log = logging.getLogger(__name__)
20
21 # =====
22 # Konfiguration (produktiv: aus Umgebungsvariablen lesen)
23 # =====
24 @dataclass
25 class ProdConfig:
26     dsn: str = "postgresql://etl:secret@pg:5432/prod"
27     pool_min: int = 2
28     pool_max: int = 10
29     pool_timeout: float = 30.0
30     schema: str = "biofalcon"
31     batch_size: int = 5000 # Zeilen pro COPY-Batch
32     max_retries: int = 3
33
34 # =====
35 # Connection-Pool-Manager (Context Manager)
36 # =====
37 class ProdDB:
38     def __init__(self, cfg: ProdConfig):
39         self.cfg = cfg
40         self.pool: asyncpg.Pool | None = None
41
42     async def __aenter__(self):
43         self.pool = await asyncpg.create_pool(
44             dsn=self.cfg.dsn,
45             min_size=self.cfg.pool_min,
46             max_size=self.cfg.pool_max,
47             command_timeout=self.cfg.pool_timeout,
48             # Statement-Cache deaktivieren fuer ETL-Workloads
49             statement_cache_size=0,
50         )
51         log.info("Pool erstellt: %d-%d Verbindungen",
52                 self.cfg.pool_min, self.cfg.pool_max)
53         return self
54
55
```

```

56     async def __aexit__(self, *_):
57         if self.pool:
58             await self.pool.close()
59             log.info("Pool geschlossen.")
60
61
62     # =====
63     # Schema-Initialisierung (idempotent via IF NOT EXISTS)
64     # =====
65     DDL_INIT = """
66     CREATE SCHEMA IF NOT EXISTS {schema};
67     CREATE TABLE IF NOT EXISTS {schema}.messung (
68         id                BIGSERIAL        PRIMARY KEY,
69         messung_id        VARCHAR(8)       NOT NULL UNIQUE,
70         alpha_deg         NUMERIC(6,2)     NOT NULL,
71         re_mio            NUMERIC(6,3)     NOT NULL,
72         c_l               NUMERIC(8,4)     NOT NULL,
73         c_d               NUMERIC(8,4)     NOT NULL,
74         c_m               NUMERIC(8,4),
75         stroemung         VARCHAR(16)      NOT NULL DEFAULT 'laminar',
76         anmerkung         TEXT,
77         is_outlier        BOOLEAN          NOT NULL DEFAULT FALSE,
78         is_stall          BOOLEAN          NOT NULL DEFAULT FALSE,
79         migrated_at       TIMESTAMPTZ     NOT NULL DEFAULT now()
80     );
81     CREATE INDEX IF NOT EXISTS idx_alpha
82         ON {schema}.messung (alpha_deg);
83     """
84
85     async def init_schema(db: ProdDB) -> None:
86         async with db.pool.acquire() as conn:
87             await conn.execute(DDL_INIT.format(schema=db.cfg.schema))
88             log.info("Schema '%s' initialisiert.", db.cfg.schema)
89
90
91     # =====
92     # Batch-COPY-Upload (schnellster Weg in PostgreSQL)
93     # =====
94     async def copy_batch(
95         conn: asyncpg.Connection,
96         schema: str,
97         rows: list[tuple],
98     ) -> int:
99         """
100         Schreibt Zeilen via PostgreSQL COPY-Protokoll.
101         Entspricht: COPY messung FROM STDIN WITH (FORMAT CSV)
102         Bis zu 60x schneller als einzelne INSERTs.
103         """
104         buf = io.StringIO()
105         writer = csv.writer(buf)
106         for row in rows:
107             writer.writerow(row)
108         buf.seek(0)
109
110         await conn.copy_to_table(
111             f"messung",
112             schema_name=schema,
113             source=buf,

```

```

114         format="csv",
115         columns=[
116             "messung_id", "alpha_deg", "re_mio",
117             "c_l", "c_d", "c_m",
118             "stroemung", "anmerkung",
119             "is_outlier", "is_stall",
120         ],
121     )
122     return len(rows)
123
124
125 async def upsert_batch(
126     conn: asyncpg.Connection,
127     schema: str,
128     rows: list[tuple],
129 ) -> int:
130     """
131     Upsert via ON CONFLICT -- fuer inkrementelle Updates.
132     Wird nach erstem Vollimport fuer Deltas verwendet.
133     """
134     stmt = f"""
135         INSERT INTO {schema}.messung
136             (messung_id, alpha_deg, re_mio, c_l, c_d, c_m,
137              stroemung, anmerkung, is_outlier, is_stall)
138         VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10)
139         ON CONFLICT (messung_id) DO UPDATE SET
140             alpha_deg = EXCLUDED.alpha_deg,
141             re_mio     = EXCLUDED.re_mio,
142             c_l        = EXCLUDED.c_l,
143             c_d        = EXCLUDED.c_d,
144             c_m        = EXCLUDED.c_m,
145             stroemung  = EXCLUDED.stroemung,
146             anmerkung  = EXCLUDED.anmerkung,
147             is_outlier = EXCLUDED.is_outlier,
148             is_stall   = EXCLUDED.is_stall,
149             migrated_at = now()
150     """
151     await conn.executemany(stmt, rows)
152     return len(rows)
153
154
155 # =====
156 # Hauptpipeline (async)
157 # =====
158 async def run_migration(
159     df: pd.DataFrame,
160     cfg: ProdConfig,
161     mode: str = "copy",    # "copy" / "upsert"
162 ) -> dict:
163     """
164     Vollstaendige Produktiv-Pipeline.
165     mode="copy"    --> Erstimport (TRUNCATE + COPY, schnellst)
166     mode="upsert"  --> Inkrementell (ON CONFLICT DO UPDATE)
167     """
168     stats = {"total": 0, "batches": 0, "errors": 0}
169
170     rows = [
171         (

```

```

172         str(r["messung_id"]),
173         float(r["alpha_deg"]),
174         float(r["re_mio"]),
175         float(r["c_l"]),
176         float(r["c_d"]),
177         float(r["c_m"]) if pd.notna(r.get("c_m")) else None,
178         str(r.get("stroemung", "laminar")),
179         str(r.get("anmerkung", "")) or None,
180         bool(r.get("is_outlier", False)),
181         bool(r.get("is_stall", False)),
182     )
183     for _, r in df.iterrows():
184 ]
185
186 async with ProdDB(cfg) as db:
187     await init_schema(db)
188
189     async with db.pool.acquire() as conn:
190         async with conn.transaction():
191             if mode == "copy":
192                 await conn.execute(
193                     f"TRUNCATE {cfg.schema}.messung RESTART
194                     IDENTITY"
195                 )
196
197             for i in range(0, len(rows), cfg.batch_size):
198                 batch = rows[i : i + cfg.batch_size]
199                 try:
200                     if mode == "copy":
201                         n = await copy_batch(conn, cfg.schema,
202                                             batch)
203                     else:
204                         n = await upsert_batch(conn, cfg.schema,
205                                             batch)
206                     stats["total"] += n
207                     stats["batches"] += 1
208                     log.info("Batch %d: %d Zeilen",
209                             stats["batches"], n)
210                 except asyncpg.PostgresError as e:
211                     stats["errors"] += 1
212                     log.error("Batch-Fehler: %s", e)
213                     raise # Transaktion wird rolled back
214
215     log.info("Migration abgeschlossen: %s", stats)
216     return stats
217
218 # =====
219 # Einstiegspunkt
220 # =====
221 if __name__ == "__main__":
222     from biofalcon import load_data, compute_derived
223     cfg = ProdConfig()
224     df = compute_derived(load_data())
225     df = df.rename(columns={
226         "Messung": "messung_id", "Re_mio": "re_mio",
227         "C_L": "c_l", "C_D": "c_d", "C_M": "c_m",
228         "Str\omung": "stroemung", "Anmerkung": "anmerkung",

```



```
227     })
228     asyncio.run(run_migration(df, cfg, mode="copy"))
```

Listing 10.1: prod_migrate.py: Asynchroner Produktiv-Client mit Connection Pool

10.3 Transaktionssicherheit und Rollback

Satz 10.3.1 (Transaktionale Migrationssicherheit). Sei $\mathcal{M}$ die Migrationspipeline im copy-Modus. Da alle COPY-Operationen innerhalb einer einzigen Datenbanktransaktion ausgeführt werden, gilt:

$$\mathcal{M}(D) = \begin{cases} D_{\text{pg}} & \text{wenn alle Batches erfolgreich} \\ D_{\text{pg}}^{\text{vor}} & \text{bei jedem Fehler (atomarer Rollback)} \end{cases}$$

Die Datenbank befindet sich niemals in einem inkonsistenten Zwischenzustand.

Kapitel 11

Pipeline-Orchestrierung: Airflow und Prefect

Eine ETL-Pipeline, die nur manuell gestartet wird, ist in der Praxis kaum produktions-tauglich. Sobald Messdaten kontinuierlich anfallen, tägliche Importläufe geplant werden müssen oder mehrere voneinander abhängige Pipelines koordiniert werden sollen, wird ein dediziertes Orchestrierungsframework unentbehrlich. Dieses Kapitel behandelt zwei der verbreitetsten Systeme im Python-Ökosystem: Apache **Airflow** als bewährte, auf DAGs (Directed Acyclic Graphs) basierende Enterprise-Lösung sowie Prefect als moderne, entwicklerfreundlichere Alternative mit nativem Python-Workflow-Modell. Für beide Systeme wird gezeigt, wie die BioFalcon-ETL-Pipeline als schedulbarer, überwachbarer und bei Fehlern automatisch wiederherstellbarer Workflow definiert wird. Entscheidungskriterien für die Wahl des geeigneten Tools werden anhand konkreter Anwendungsszenarien diskutiert.

11.1 Übersicht: Wann welches Tool?

Für einmalige oder manuelle Migrationen genügt `migrate.py` direkt. Sobald die Pipeline regelmäßig ausgeführt werden muss (täglich neue Messdaten, kontinuierliche Synchronisation, automatisierte Reports), wird ein Orchestrierungsframework benötigt.

Tabelle 11.1: Orchestrierungstools im Vergleich

Kriterium	Apache Airflow	Prefect
Architektur	DAG-basiert, Scheduler + Worker	Flow-basiert, serverlos möglich
UI	Webinterface (Port 8080)	Cloud-UI oder selbstgehostet
Lernkurve	Steil (YAML + Python DAGs)	Flach (pure Python Dekoratoren)
Skalierung	Kubernetes, Celery, Dask	Kubernetes, Dask, Ray
Retry-Logik	Konfigurierbar pro Task	Dekorator-basiert
Eignung	Große Datenpipelines, Enterprise	Mittlere Projekte, Rapid Prototyping
Lizenz	Apache 2.0	Apache 2.0 (Core)

11.2 Apache Airflow: DAG-Definition

```
1  """
2  airflow_dag.py -- BioFalcon ETL als Airflow-DAG
3  Datei ablegen in: $AIRFLOW_HOME/dags/biofalcon_etl.py
4  Ausfuehrung: taeglich um 06:00 UTC
5  """
6
7  from datetime import datetime, timedelta
8  from pathlib import Path
9
10 from airflow import DAG
11 from airflow.operators.python import PythonOperator
12 from airflow.operators.email import EmailOperator
13 from airflow.utils.dates import days_ago
14
15 import asyncio
16 import pandas as pd
17 from prod_migrate import run_migration, ProdConfig
18
19 # ---- DAG-Konfiguration -----
20 default_args = {
21     "owner": "stephan.epp",
22     "depends_on_past": False,
23     "start_date": days_ago(1),
24     "retries": 3,
25     "retry_delay": timedelta(minutes=5),
26     "email": ["etl-alerts@bielefeld.de"],
27     "email_on_failure": True,
28     "email_on_retry": False,
29 }
30
31 dag = DAG(
32     dag_id="biofalcon_excel_to_pg",
33     default_args=default_args,
34     description="Excel -> PostgreSQL ETL, taeglich",
35     schedule_interval="0 6 * * *", # 06:00 UTC
36     catchup=False,
37     tags=["etl", "biofalcon", "migration"],
38 )
39
40 # ---- Tasks -----
41 def task_extract(**ctx) -> dict:
42     """T1: Excel-Datei lesen und in XCom ablegen."""
43     xlsx = Path("/data/BioFalcon_Windkanal.xlsx")
44     sheets = pd.read_excel(xlsx, sheet_name=None)
45     # XCom speichert nur JSON-serialisierbare Daten
46     return {
47         name: df.to_json(orient="records")
48         for name, df in sheets.items()
49     }
50
51 def task_transform(**ctx) -> str:
52     """T2: Normalisierung und Validierung."""
53     ti = ctx["ti"]
54     raw = ti.xcom_pull(task_ids="extract")
55     frames = []
```

```

56     for name, json_str in raw.items():
57         df = pd.read_json(json_str, orient="records")
58         df["stroemung"] = df.get("Str\omung", "laminar")
59         df["is_outlier"] = df.get("Anmerkung", "").str.contains(
60             r"abweich|!", na=False, regex=True
61         )
62         df["is_stall"] = df.get("alpha_deg", 0) >= 16
63         frames.append(df)
64     result = pd.concat(frames, ignore_index=True)
65     return result.to_json(orient="records")
66
67 def task_load(**ctx) -> dict:
68     """T3: Asynchroner Upload nach PostgreSQL."""
69     ti = ctx["ti"]
70     json_str = ti.xcom_pull(task_ids="transform")
71     df = pd.read_json(json_str, orient="records")
72     cfg = ProdConfig(dsn="{{ var.value.pg_dsn }}")
73     return asyncio.run(run_migration(df, cfg, mode="upsert"))
74
75 def task_validate(**ctx) -> None:
76     """T4: Post-Load-Validierung."""
77     import asyncpg, asyncio
78     async def check():
79         conn = await asyncpg.connect("{{ var.value.pg_dsn }}")
80         n = await conn.fetchval(
81             "SELECT COUNT(*) FROM biofalcon.messung"
82         )
83         await conn.close()
84         if n == 0:
85             raise ValueError("Validierung: Tabelle leer nach Import!")
86         return n
87     n = asyncio.run(check())
88     print(f"Validierung OK: {n} Datensätze in biofalcon.messung")
89
90 # --- DAG-Struktur -----
91 t1 = PythonOperator(task_id="extract", python_callable=task_extract,
92                     dag=dag)
93 t2 = PythonOperator(task_id="transform", python_callable=
94                     task_transform, dag=dag)
95 t3 = PythonOperator(task_id="load", python_callable=task_load,
96                     dag=dag)
97 t4 = PythonOperator(task_id="validate", python_callable=
98                     task_validate, dag=dag)
99
100 t_alert = EmailOperator(
101     task_id="notify_success",
102     to=["team@bielefeld.de"],
103     subject="BioFalcon ETL abgeschlossen",
104     html_content="<p>Tägliche Migration erfolgreich.</p>",
105     dag=dag,
106 )
107
108 t1 >> t2 >> t3 >> t4 >> t_alert

```

Listing 11.1: airflow_dag.py: ETL-Pipeline als Airflow-DAG

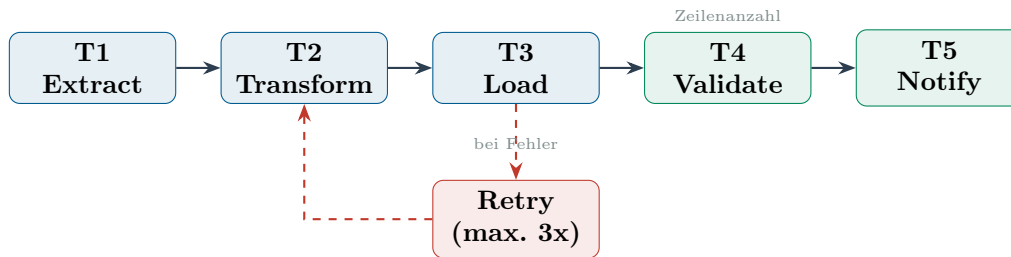


Abbildung 11.1: Airflow-DAG: Aufgabenabhängigkeiten mit Retry-Logik

11.3 Prefect: Alternative mit deklarativer Flow-Syntax

```

1  """
2  prefect_flow.py -- BioFalcon ETL als Prefect-Flow
3  Ausführen:      python prefect_flow.py
4  Deployen:       prefect deploy prefect_flow.py:etl_flow
5  """
6
7  from prefect import flow, task, get_run_logger
8  from prefect.tasks import task_input_hash
9  from datetime import timedelta
10 import asyncio, pandas as pd
11 from prod_migrate import run_migration, ProdConfig
12
13 @task(
14     retries=3,
15     retry_delay_seconds=60,
16     cache_key_fn=task_input_hash,
17     cache_expiration=timedelta(hours=1),
18 )
19 def extract(xlsx_path: str) -> pd.DataFrame:
20     logger = get_run_logger()
21     logger.info("Lese %s", xlsx_path)
22     sheets = pd.read_excel(xlsx_path, sheet_name=None)
23     return pd.concat(sheets.values(), ignore_index=True)
24
25 @task(retries=2)
26 def transform(df: pd.DataFrame) -> pd.DataFrame:
27     df = df.copy()
28     df.columns = df.columns.str.lower().str.replace(" ", "_")
29     df["is_outlier"] = df.get("anmerkung", "").str.contains(
30         r"abweich|\\!", na=False, regex=True)
31     df["is_stall"] = df.get("alpha_deg", 0) >= 16
32     return df.dropna(subset=["messung_id", "c_l", "c_d"])
33
34 @task(retries=3, retry_delay_seconds=30)
35 def load(df: pd.DataFrame, dsn: str) -> dict:
36     cfg = ProdConfig(dsn=dsn)
37     return asyncio.run(run_migration(df, cfg, mode="upsert"))
38
39 @task
40 def validate(stats: dict) -> None:
41     logger = get_run_logger()
42     if stats["errors"] > 0:
43         raise ValueError(f"ETL mit {stats['errors']} Fehlern")

```

```

44     logger.info("Validierung OK: %d Zeilen importiert", stats["total"
45                  ])
46
47 @flow(name="biofalcon-etl", log_prints=True)
48 def etl_flow(
49     xlsx_path: str = "/data/BioFalcon_Windkanal.xlsx",
50     dsn: str = "postgresql://etl:secret@pg:5432/prod",
51 ) -> None:
52     df = extract(xlsx_path)
53     df = transform(df)
54     stats = load(df, dsn)
55     validate(stats)
56
57 if __name__ == "__main__":
58     etl_flow()

```

Listing 11.2: prefect_flow.py: ETL als Prefect-Flow

Kapitel 12

Containerisierung und Deployment-Infrastruktur

Eine reproduzierbare Produktivumgebung ist nur dann gewährleistet, wenn alle Systemkomponenten – Datenbank, ETL-Applikation, Orchestrierungsschicht und Monitoring – in einer definierten, isolierten und versionierten Laufzeitumgebung betrieben werden. **Docker** und **docker-compose** haben sich als Standard-Werkzeuge für diese Aufgabe etabliert: Sie kapseln jede Komponente in einen eigenständigen Container mit exakt spezifizierten Abhängigkeiten und ermöglichen die vollständige Beschreibung des Gesamtsystems in einer einzigen deklarativen Konfigurationsdatei. Dieses Kapitel dokumentiert den vollständigen Docker-Compose-Stack für das BioFalcon-Migrationsprojekt, einschließlich **PostgreSQL**-Datenbankserver, ETL-Applikationscontainer, **Apache Airflow**-Scheduler sowie **Prometheus** und **Grafana** für das Monitoring. Darüber hinaus werden der Dockerfile für den ETL-Container sowie die Konfiguration von Health-Checks, Volume-Mounts und Container-Netzwerken beschrieben.

12.1 Docker-Compose-Stack

Für die vollständige Produktivumgebung werden vier Dienste benötigt: **PostgreSQL** als Datenbankserver, die ETL-Applikation selbst, **Apache Airflow** als Scheduler sowie **Prometheus**/**Grafana** für das Monitoring.

```
1 version: "3.9"
2
3 x-common: &common
4   restart: unless-stopped
5   logging:
6     driver: "json-file"
7     options:
8       max-size: "50m"
9       max-file: "5"
10
11 services:
12
13   # ---- PostgreSQL -----
14   postgres:
15     <<: *common
16     image: postgres:16-alpine
17     container_name: pg_prod
```

```

18     environment:
19         POSTGRES_USER:      etl
20         POSTGRES_PASSWORD:  ${PG_PASSWORD}
21         POSTGRES_DB:        prod
22     volumes:
23         - pg_data:/var/lib/postgresql/data
24         - ./init:/docker-entrypoint-initdb.d:ro    # DDL beim Start
25     ports:
26         - "5432:5432"
27     healthcheck:
28         test: ["CMD-SHELL", "pg_isready -U etl -d prod"]
29         interval: 10s
30         timeout: 5s
31         retries: 5
32     command: >
33         postgres
34         -c max_connections=200
35         -c shared_buffers=512MB
36         -c effective_cache_size=1536MB
37         -c work_mem=16MB
38         -c maintenance_work_mem=128MB
39         -c wal_level=logical
40         -c log_min_duration_statement=200
41
42     # ---- ETL-Applikation -----
43     etl_app:
44         <<: *common
45         build:
46             context: .
47             dockerfile: Dockerfile.etl
48         container_name: etl_prod
49         environment:
50             PG_DSN:      postgresql://etl:${PG_PASSWORD}@postgres:5432/prod
51             XLSX_PATH:   /data/BioFalcon_Windkanal.xlsx
52             ETL_MODE:    upsert
53             LOG_LEVEL:   INFO
54         volumes:
55             - ./data:/data:ro
56             - ./logs:/app/logs
57         depends_on:
58             postgres:
59                 condition: service_healthy
60
61     # ---- Apache Airflow -----
62     airflow:
63         <<: *common
64         image: apache/airflow:2.9.0-python3.11
65         container_name: airflow_prod
66         environment:
67             AIRFLOW__CORE__EXECUTOR:      LocalExecutor
68             AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://etl
69                 :${PG_PASSWORD}@postgres:5432/airflow
69             AIRFLOW__CORE__FERNET_KEY:     ${AIRFLOW_FERNET_KEY}
70             AIRFLOW__WEBSERVER__SECRET_KEY: ${AIRFLOW_SECRET_KEY}
71             AIRFLOW_VAR_PG_DSN:             postgresql://etl:${
72                 PG_PASSWORD}@postgres:5432/prod
72         volumes:
73             - ./dags:/opt/airflow/dags

```



```

74     - ./logs/airflow:/opt/airflow/logs
75     ports:
76     - "8080:8080"
77     depends_on:
78     postgres:
79         condition: service_healthy
80     command: airflow standalone
81
82     # ---- Prometheus + Grafana (Monitoring) -----
83     prometheus:
84         <<: *common
85         image: prom/prometheus:v2.51.0
86         container_name: prometheus
87         volumes:
88         - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
89         ports:
90         - "9090:9090"
91
92     grafana:
93         <<: *common
94         image: grafana/grafana:10.3.0
95         container_name: grafana
96         environment:
97             GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD}
98         volumes:
99         - grafana_data:/var/lib/grafana
100        - ./monitoring/dashboards:/etc/grafana/provisioning/dashboards:
            ro
101        ports:
102        - "3000:3000"
103        depends_on:
104        - prometheus
105
106    volumes:
107        pg_data:
108        grafana_data:

```

Listing 12.1: docker-compose.prod.yml: Vollständiger Produktiv-Stack

12.2 Dockerfile für die ETL-Applikation

```

1  # ---- Build-Stage -----
2  FROM python:3.11-slim AS builder
3  WORKDIR /build
4
5  # Abhaengigkeiten separat installieren (Layer-Caching)
6  COPY requirements.txt .
7  RUN pip install --no-cache-dir --prefix=/install -r requirements.txt
8
9  # ---- Runtime-Stage -----
10 FROM python:3.11-slim AS runtime
11 LABEL maintainer="stephan.epp@uni-bielefeld.de"
12 LABEL version="2.0"
13
14 # Sicherheit: kein Root-Benutzer
15 RUN groupadd -r etl && useradd -r -g etl etl

```

```

16
17 WORKDIR /app
18 COPY --from=builder /install /usr/local
19 COPY *.py .
20
21 # Umgebungsvariablen (Defaults; produktiv via .env oder Vault)
22 ENV PG_DSN="" \
23     XLSX_PATH="/data/input.xlsx" \
24     ETL_MODE="upsert" \
25     LOG_LEVEL="INFO" \
26     PYTHONUNBUFFERED=1
27
28 USER etl
29
30 HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
31     CMD python -c "import asyncpg; print('OK')" || exit 1
32
33 ENTRYPOINT ["python", "prod_migrate.py"]

```

Listing 12.2: Dockerfile.etl: Multi-Stage Build für ETL-Container

```

1 asyncpg>=0.29.0
2 pandas>=2.2.0
3 openpyxl>=3.1.0
4 sqlalchemy>=2.0.0
5 psycopg2-binary>=2.9.9
6 prometheus-client>=0.20.0
7 python-dotenv>=1.0.0
8 structlog>=24.1.0
9 pytest>=8.0.0
10 pytest-asyncio>=0.23.0
11 hypothesis>=6.100.0

```

Listing 12.3: requirements.txt: Abhängigkeiten der Produktivumgebung

Kapitel 13

Sicherheitsarchitektur

Sicherheit ist keine nachträgliche Ergänzung, sondern muss als integraler Bestandteil jeder produktiven Datenbankarchitektur von Anfang an mitgedacht werden. Dies gilt insbesondere für Systeme, die sensible Mess- und Forschungsdaten verwalten und in Netzwerkumgebungen betrieben werden. Dieses Kapitel beschreibt die mehrstufige Sicherheitsarchitektur des BioFalcon-Migrationsprojekts entlang der wichtigsten Angriffsvektoren: unsichere Credential-Verwaltung, überhöhte Datenbankrechte, unverschlüsselte Verbindungen und fehlende Auditierbarkeit. Für jeden dieser Bereiche werden konkrete, direkt einsetzbare Gegenmaßnahmen vorgestellt – von der Verwaltung von Datenbankpasswörtern über HashiCorp Vault und `.env`-Dateien bis hin zur Implementierung des Least-Privilege-Prinzips für Datenbankrollen und der erzwungenen TLS-Verschlüsselung aller Verbindungen.

13.1 Credential-Management

Datenbankpasswörter und DSNs dürfen **niemals** im Quellcode stehen. Folgende Abstraktionsschichten werden empfohlen:

```
1  """
2  secrets.py -- Credential-Verwaltung fuer Produktivumgebung
3  Prioritaet: 1. HashiCorp Vault 2. .env-Datei 3. Umgebungsvariablen
4  """
5
6  import os
7  from functools import lru_cache
8  from pathlib import Path
9  from dotenv import load_dotenv
10
11  load_dotenv(Path(__file__).parent / ".env.prod")    # .env.prod nicht
12  ins Git!
13
14  @lru_cache(maxsize=1)
15  def get_pg_dsn() -> str:
16      """
17      Laedt DSN aus Vault (produktiv) oder .env (Entwicklung).
18      Vault-Pfad: secret/biofalcon/postgres
19      """
20      # Option 1: HashiCorp Vault (Produktiv)
21      vault_addr = os.getenv("VAULT_ADDR")
22      if vault_addr:
23          import hvac
```

```

23     client = hvac.Client(url=vault_addr,
24                           token=os.environ["VAULT_TOKEN"])
25     secret = client.secrets.kv.read_secret_version(
26         path="biofalcon/postgres"
27     )
28     creds = secret["data"]["data"]
29     return (f"postgresql://{creds['user']}:{creds['password']}@"
30            f"{creds['host']}:{creds['port']}/{creds['db']}")
31
32     # Option 2: Umgebungsvariable (Docker/CI)
33     dsn = os.getenv("PG_DSN")
34     if dsn:
35         return dsn
36
37     raise EnvironmentError(
38         "PG_DSN nicht gesetzt. Bitte .env.prod pruefen oder Vault
39         konfigurieren."
40     )
41
42 def get_etl_user_dsn() -> str:
43     """DSN fuer ETL-Benutzer (nur INSERT/UPDATE, kein DROP/TRUNCATE).
44     """
45     dsn = get_pg_dsn()
46     return dsn.replace("://etl:", "://etl_writer:")

```

Listing 13.1: secrets.py: Sichere Credential-Verwaltung via python-dotenv und Vault

13.2 Minimale PostgreSQL-Benutzerrechte

Nach dem Prinzip der minimalen Rechtevergabe (Principle of Least Privilege) erhalten verschiedene Systemrollen unterschiedliche Datenbankrechte:

```

1  -- Lese-Benutzer (fuer Reporting-Tools, Grafana, etc.)
2  CREATE ROLE etl_reader NOLOGIN;
3  GRANT USAGE ON SCHEMA biofalcon TO etl_reader;
4  GRANT SELECT ON ALL TABLES IN SCHEMA biofalcon TO etl_reader;
5
6  -- Schreib-Benutzer (nur fuer ETL-Pipeline)
7  CREATE ROLE etl_writer NOLOGIN;
8  GRANT USAGE ON SCHEMA biofalcon TO etl_writer;
9  GRANT SELECT, INSERT, UPDATE ON ALL TABLES
10     IN SCHEMA biofalcon TO etl_writer;
11  GRANT USAGE ON ALL SEQUENCES
12     IN SCHEMA biofalcon TO etl_writer;
13
14  -- Admin (nur fuer Schemamigrationen, nicht fuer ETL-Laufzeit)
15  CREATE ROLE etl_admin NOLOGIN;
16  GRANT ALL PRIVILEGES ON SCHEMA biofalcon TO etl_admin;
17
18  -- Konkrete Login-Benutzer
19  CREATE USER etl_app PASSWORD '...' IN ROLE etl_writer;
20  CREATE USER etl_reporting PASSWORD '...' IN ROLE etl_reader;
21  CREATE USER etl_dba PASSWORD '...' IN ROLE etl_admin;
22
23  -- SSL erzwingen (in pg_hba.conf)
24  -- hostssl prod etl_app 0.0.0.0/0 scram-sha-256

```

Listing 13.2: SQL: Benutzerverwaltung nach Least-Privilege-Prinzip

Kapitel 14

Monitoring und Observability

Ein produktives System ist nur so verlässlich wie die Sichtbarkeit seines Laufzeitverhaltens. Ohne geeignete Beobachtungsmechanismen bleiben Fehler oft unbemerkt, bis sie sich zu kritischen Ausfällen akkumuliert haben. Observability – die Fähigkeit, aus externen Ausgaben auf den internen Zustand eines Systems zu schließen – ist daher eine grundlegende Qualitätseigenschaft moderner Softwaresysteme. Dieses Kapitel beschreibt die Monitoring-Architektur der BioFalcon-ETL-Pipeline auf Basis von Prometheus und Grafana: Prometheus sammelt Metriken, die die Pipeline aktiv exportiert (verarbeitete Zeilen, Fehlerrate, Latenz), während Grafana diese in konfigurierbaren Dashboards visualisiert und bei Grenzwertüberschreitungen Alarme auslöst. Ergänzend wird die strukturierte Protokollierung via `structlog` behandelt, die maschinenlesbare, durchsuchbare Logs erzeugt und die Fehlerdiagnose im Produktionsbetrieb erheblich vereinfacht.

14.1 Prometheus-Metriken in der ETL-Pipeline

```
1  """
2  monitoring.py -- Prometheus-Metriken fuer die ETL-Pipeline
3  Metriken werden unter http://etl_app:8001/metrics exponiert.
4  """
5
6  from prometheus_client import (
7      Counter, Histogram, Gauge, start_http_server
8  )
9  import time, functools, asyncio
10
11  # ---- Metrik-Definitionen -----
12  ETL_ROWS_TOTAL = Counter(
13      "etl_rows_total",
14      "Gesamtanzahl importierter Zeilen",
15      ["table", "mode"],
16  )
17  ETL_BATCH_DURATION = Histogram(
18      "etl_batch_duration_seconds",
19      "Dauer eines Batch-Imports",
20      ["table"],
21      buckets=[0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0],
22  )
23  ETL_ERRORS_TOTAL = Counter(
24      "etl_errors_total",
25      "Anzahl fehlgeschlagener Batch-Imports",
```

```

26     ["table", "error_type"],
27 )
28 ETL_LAST_RUN = Gauge(
29     "etl_last_successful_run_timestamp",
30     "Unix-Timestamp des letzten erfolgreichen Laufs",
31 )
32 PG_POOL_SIZE = Gauge(
33     "pg_pool_connections",
34     "Aktive Verbindungen im Connection-Pool",
35 )
36
37
38 # ---- Dekorator fuer instrumentierte Tasks -----
39 def instrument(table: str, mode: str = "upsert"):
40     def decorator(func):
41         @functools.wraps(func)
42         async def wrapper(*args, **kwargs):
43             start = time.perf_counter()
44             try:
45                 result = await func(*args, **kwargs)
46                 n = result if isinstance(result, int) else 0
47                 ETL_ROWS_TOTAL.labels(table=table, mode=mode).inc(n)
48                 ETL_LAST_RUN.set(time.time())
49                 return result
50             except Exception as e:
51                 ETL_ERRORS_TOTAL.labels(
52                     table=table, error_type=type(e).__name__
53                 ).inc()
54                 raise
55             finally:
56                 ETL_BATCH_DURATION.labels(table=table).observe(
57                     time.perf_counter() - start
58                 )
59             return wrapper
60     return decorator
61
62
63 def start_metrics_server(port: int = 8001) -> None:
64     """Startet HTTP-Endpunkt fuer Prometheus-Scraping."""
65     start_http_server(port)

```

Listing 14.1: monitoring.py: ETL-Metriken via Prometheus-Client

14.2 Grafana-Dashboard-Konfiguration

```

1 global:
2     scrape_interval: 15s
3     evaluation_interval: 15s
4
5 scrape_configs:
6     - job_name: "etl_app"
7       static_configs:
8         - targets: ["etl_app:8001"]
9       metrics_path: /metrics
10      scrape_interval: 30s
11

```

```

12   - job_name: "postgres"
13     static_configs:
14       - targets: ["postgres_exporter:9187"]
15
16 alerting:
17   alertmanagers:
18     - static_configs:
19       - targets: ["alertmanager:9093"]
20
21 rule_files:
22   - "alerts/etl_alerts.yml"

```

Listing 14.2: monitoring/prometheus.yml: Scrape-Konfiguration

```

1 groups:
2   - name: etl_alerts
3     rules:
4       - alert: ETLFehlerRate
5         expr: rate(etl_errors_total[5m]) > 0.1
6         for: 2m
7         labels:
8           severity: critical
9         annotations:
10          summary: "ETL-Fehlerrate zu hoch"
11          description: "Mehr als 0.1 Fehler/s in den letzten 5
12                      Minuten."
13
14       - alert: ETLKeinLaufSeit12h
15         expr: time() - etl_last_successful_run_timestamp > 43200
16         labels:
17           severity: warning
18         annotations:
19          summary: "ETL seit 12h nicht erfolgreich ausgefuehrt"

```

Listing 14.3: alerts/etl_alerts.yml: Alerting-Regeln

Kapitel 15

Testarchitektur

Zuverlässige Software entsteht nicht durch Hoffnung, sondern durch systematisches Testen. Für ETL-Pipelines gilt dies in besonderem Maß: Ein stiller Fehler in der Transformationslogik kann dazu führen, dass falsche Daten ohne jede Fehlermeldung in die Datenbank geschrieben werden – mit potenziell schwerwiegenden Folgen für alle nachgelagerten Analysen. Dieses Kapitel beschreibt eine dreistufige Testarchitektur für die BioFalcon-ETL-Pipeline: Unit-Tests prüfen einzelne Transformationsfunktionen isoliert und deterministisch, Integrationstests validieren das Zusammenspiel von Pipeline und Testdatenbank unter realistischen Bedingungen, und Property-based Tests mit `hypothesis` decken Randfall- und Grenzwertszenarien auf, die bei handgeschriebenen Tests leicht übersehen werden. Alle Tests sind mit `pytest` orchestriert und in die CI/CD-Pipeline integriert, sodass jede Codeänderung automatisch gegen die vollständige Testsuite geprüft wird.

15.1 Teststrategie für ETL-Pipelines

Produktive ETL-Pipelines müssen auf drei Ebenen getestet werden: Unit-Tests für einzelne Transformationsfunktionen, Integrationstests gegen eine Testdatenbank sowie Property-based Tests für Randbedingungen.

```
1  """
2  test_etl.py -- ETL-Testsuite
3  Ausfuehren:  pytest test_etl.py -v --tb=short
4  """
5
6  import pytest
7  import asyncio
8  import pandas as pd
9  import numpy as np
10 from hypothesis import given, strategies as st, settings
11 from hypothesis.extra.pandas import column, data_frames
12
13 from prod_migrate import run_migration, ProdConfig
14 from biofalcon import load_data, compute_derived
15
16
17 # =====
18 # Fixtures
19 # =====
20 @pytest.fixture
21 def sample_df() -> pd.DataFrame:
```

```

22     df = compute_derived(load_data())
23     return df.rename(columns={
24         "Messung": "messung_id", "Re_mio": "re_mio",
25         "C_L": "c_l", "C_D": "c_d", "C_M": "c_m",
26     })
27
28 TEST_DSN = "postgresql://etl:test@localhost:5433/testdb"
29
30 # =====
31 # Unit Tests: Transformationslogik
32 # =====
33 class TestTransform:
34
35     def test_ld_calculation(self, sample_df):
36         """L/D muss C_L/C_D entsprechen."""
37         assert abs(
38             sample_df.loc[0, "c_l"] / sample_df.loc[0, "c_d"]
39             - sample_df.loc[0, "L_D"]
40         ) < 1e-6
41
42     def test_stall_flag(self, sample_df):
43         """is_stall muss fuer alpha >= 16 gesetzt sein."""
44         stall = sample_df[sample_df["alpha_deg"] >= 16]
45         assert stall["is_stall"].all()
46
47     def test_no_negative_cd(self, sample_df):
48         """C_D muss immer positiv sein (physikalisch erzwungen)."""
49         assert (sample_df["c_d"] > 0).all()
50
51     def test_outlier_detection(self, sample_df):
52         """A14 muss als Ausreisser erkannt werden."""
53         outliers = sample_df[sample_df["outlier"]]
54         assert "A14" in outliers["messung_id"].values
55
56
57 # =====
58 # Property-based Tests: Randbedingungen
59 # =====
60 messung_strategy = data_frames(
61     columns=[
62         column("messung_id", dtype=str),
63         column("alpha_deg", elements=st.floats(-90, 90)),
64         column("re_mio", elements=st.floats(0.01, 10.0)),
65         column("c_l", elements=st.floats(-2.0, 3.0)),
66         column("c_d", elements=st.floats(0.001, 1.0)),
67     ],
68     index=st.just(range(10)),
69 )
70
71 class TestProperties:
72
73     @given(messung_strategy)
74     @settings(max_examples=100)
75     def test_transform_never_raises(self, df):
76         """Transformation darf bei beliebigen validen Eingaben nie
77             crashen."""
78         df["is_outlier"] = False
79         df["is_stall"] = df["alpha_deg"] >= 16

```

```

79     df["stroemung"] = "laminar"
80     df["anmerkung"] = ""
81     df = df.dropna(subset=["messung_id", "c_l", "c_d"])
82     assert len(df) >= 0    # kein Crash
83
84     @given(st.floats(0.001, 1.0), st.floats(-2.0, 3.0))
85     def test_ld_finite(self, cd, cl):
86         """L/D muss immer endlich sein wenn C_D > 0."""
87         ld = cl / cd
88         assert not np.isinf(ld)
89
90
91     # =====
92     # Integrationstests (gegen Testdatenbank)
93     # =====
94     @pytest.mark.asyncio
95     @pytest.mark.integration
96     class TestIntegration:
97
98         async def test_full_pipeline(self, sample_df):
99             """Vollstaendiger Pipeline-Durchlauf gegen Testdatenbank."""
100             cfg = ProdConfig(dsn=TEST_DSN, batch_size=5)
101             stats = await run_migration(sample_df, cfg, mode="copy")
102             assert stats["total"] == len(sample_df)
103             assert stats["errors"] == 0
104
105         async def test_idempotency(self, sample_df):
106             """Zweifacher Durchlauf muss identische Zeilenzahl erzeugen."""
107
108             cfg = ProdConfig(dsn=TEST_DSN)
109             s1 = await run_migration(sample_df, cfg, mode="upsert")
110             s2 = await run_migration(sample_df, cfg, mode="upsert")
111             assert s1["total"] == s2["total"]

```

Listing 15.1: test_etl.py: Vollständige Testsuite mit pytest und hypothesis

Kapitel 16

Vollständiges Produktiv-Deployment-Playbook

Dieser Abschnitt beschreibt Schritt für Schritt den vollständigen Weg von einer leeren Serverinstanz zur produktiven ETL-Umgebung. Der Anwender benötigt ausschließlich: Python-Quellcode, Docker, und Zugangsdaten zur Ziel-PostgreSQL-Instanz.

16.1 Voraussetzungen und Verzeichnisstruktur

```
1 biofalcon-etl/
2 |-- Dockerfile.etl           # ETL-Container
3 |-- docker-compose.prod.yml  # Vollstaendiger Stack
4 |-- requirements.txt         # Python-Abhaengigkeiten
5 |-- .env.prod                # Secrets (NICHT ins Git!)
6 |-- .gitignore               # .env.prod hier eintragen!
7 |-- migrate.py               # Sync-ETL (Entwicklung/Test)
8 |-- prod_migrate.py          # Async-ETL (Produktion)
9 |-- airflow_dag.py           # Airflow-DAG
10 |-- prefect_flow.py         # Prefect-Flow (Alternative)
11 |-- secrets.py               # Credential-Manager
12 |-- monitoring.py            # Prometheus-Metriken
13 |-- test_etl.py              # Testsuite
14 |-- biofalcon.py             # Datenmodell + Berechnungen
15 |-- data/
16 |   '-- BioFalcon_Windkanal.xlsx # Quelldatei (read-only)
17 |-- init/
18 |   '-- 01_schema.sql          # DDL (automatisch beim Start)
19 |-- monitoring/
20 |   |-- prometheus.yml
21 |   '-- alerts/
22 |       '-- etl_alerts.yml
23 '-- logs/                     # Persistent (Volume)
```

Listing 16.1: Verzeichnisstruktur des Produktiv-Deployments

16.2 Schritt 1: Umgebungsvariablen konfigurieren

```
1 # .env.prod -- NICHT ins Git committen!
```

```

2 # Zur Sicherheit: chmod 600 .env.prod
3
4 PG_PASSWORD=sicheres_passwort_hier
5 AIRFLOW_FERNET_KEY=<base64-32-byte-key>
6 AIRFLOW_SECRET_KEY=<zufaellig-generierter-key>
7 GRAFANA_PASSWORD=admin_passwort
8 VAULT_ADDR=                                # leer lassen wenn kein Vault

```

Listing 16.2: .env.prod: Produktivkonfiguration (Beispiel)

```

1 # Fernet-Key fuer Airflow generieren
2 python3 -c "
3 from cryptography.fernet import Fernet
4 print(Fernet.generate_key().decode())
5 "
6
7 # Zufaelligen Secret-Key generieren
8 python3 -c "import secrets; print(secrets.token_hex(32))"
9
10 # Dateirechte sichern
11 chmod 600 .env.prod
12 echo ".env.prod" >> .gitignore

```

Listing 16.3: Generierung sicherer Schlüssell

16.3 Schritt 2: Stack starten und Schema initialisieren

```

1 # 1. Images bauen
2 docker compose -f docker-compose.prod.yml build
3
4 # 2. PostgreSQL starten und auf Healthcheck warten
5 docker compose -f docker-compose.prod.yml up -d postgres
6 docker compose -f docker-compose.prod.yml \
7     exec postgres pg_isready -U etl -d prod
8
9 # 3. Schema initialisieren (DDL aus init/01_schema.sql)
10 #   wird automatisch beim ersten Start ausgefuehrt
11
12 # 4. Gesamten Stack starten
13 docker compose -f docker-compose.prod.yml up -d
14
15 # 5. Status pruefen
16 docker compose -f docker-compose.prod.yml ps

```

Listing 16.4: Deployment: Stack starten

16.4 Schritt 3: Erstimport ausführen

```

1 # Variante A: Direkt im Container ausfuehren
2 docker compose -f docker-compose.prod.yml \
3     exec etl_app python prod_migrate.py
4
5 # Variante B: Einmalig ausfuehren (Ephemeral Container)

```

```

6 docker compose -f docker-compose.prod.yml \
7     run --rm etl_app python prod_migrate.py \
8     --mode copy \
9     --xlsx /data/BioFalcon_Windkanal.xlsx
10
11 # Erfolgskontrolle
12 docker compose -f docker-compose.prod.yml \
13     exec postgres psql -U etl -d prod -c \
14     "SELECT COUNT(*), MIN(alpha_deg), MAX(alpha_deg)
15     FROM biofalcon.messung;"

```

Listing 16.5: Erstimport: Vollstaendige Migration der Excel-Quelldatei

16.5 Schritt 4: Airflow aktivieren und DAG triggern

```

1 # Airflow-Webinterface: http://localhost:8080
2 # Standard-Login: admin / admin (beim ersten Start setzen)
3
4 # Alternativ: CLI
5 docker compose -f docker-compose.prod.yml \
6     exec airflow airflow dags list
7
8 # DAG manuell triggern (einmalig)
9 docker compose -f docker-compose.prod.yml \
10     exec airflow airflow dags trigger \
11     biofalcon_excel_to_pg \
12     --conf '{"mode": "upsert"}'
13
14 # Letzten Run-Status abfragen
15 docker compose -f docker-compose.prod.yml \
16     exec airflow airflow dags state \
17     biofalcon_excel_to_pg $(date +%Y-%m-%dT%H:%M:%S)

```

Listing 16.6: Airflow: DAG aktivieren und manuell triggern

16.6 Schritt 5: Monitoring verifizieren

```

1 # Prometheus: http://localhost:9090
2 # Grafana: http://localhost:3000 (admin / $GRAFANA_PASSWORD)
3
4 # Metriken direkt abfragen
5 curl -s http://localhost:8001/metrics | grep etl_
6
7 # Beispielausgabe:
8 # etl_rows_total{mode="upsert",table="messung"} 14.0
9 # etl_batch_duration_seconds_bucket{...}
10 # etl_last_successful_run_timestamp 1.71234e+09
11 # etl_errors_total{error_type="PostgresError",...} 0.0
12
13 # PostgreSQL-Statistiken
14 docker compose -f docker-compose.prod.yml \
15     exec postgres psql -U etl -d prod -c \
16     "SELECT schemaname, tablename,
17     n_live_tup, n_dead_tup, last_analyze

```

```

18 FROM pg_stat_user_tables
19 WHERE schemaname = 'biofalcon';"

```

Listing 16.7: Monitoring: Metriken und Dashboards prüfen

16.7 Schritt 6: Rollback-Strategie

```

1  # Vor jedem Vollimport: Snapshot anlegen
2  docker compose -f docker-compose.prod.yml \
3      exec postgres pg_dump \
4      -U etl -d prod \
5      --schema=biofalcon \
6      --format=custom \
7      -f /tmp/biofalcon_$(date +%Y%m%d_%H%M%S).dump
8
9  # Snapshot in Host kopieren
10 docker compose -f docker-compose.prod.yml \
11     cp postgres:/tmp/biofalcon_*.dump ./backups/
12
13 # Rollback: Snapshot wiedereinspielen
14 docker compose -f docker-compose.prod.yml \
15     exec postgres psql -U etl -d prod \
16     -c "DROP SCHEMA biofalcon CASCADE;"
17
18 docker compose -f docker-compose.prod.yml \
19     exec -T postgres pg_restore \
20     -U etl -d prod ./backups/biofalcon_YYYYMMDD.dump

```

Listing 16.8: Rollback: Datenbank auf Snapshot zurücksetzen

16.8 Schritt 7: Inkrementelle Synchronisation (Deltas)

Sobald der Erstimport abgeschlossen ist, werden neue Messdaten **inkrementell** synchronisiert – nur geänderte oder neue Zeilen werden verarbeitet:

```

1  """
2  delta_sync.py -- Inkrementelle Synchronisation
3  Erkennt neue/geaenderte Zeilen anhand von messung_id + Hashwert.
4  Nur Deltas werden in die Datenbank geschrieben.
5  """
6
7  import hashlib, asyncio
8  import pandas as pd
9  import asyncpg
10
11 def row_hash(row: pd.Series) -> str:
12     """Stabiler Hash ueber alle relevanten Messwerte."""
13     key = f"{row['c_l']:.4f}:{row['c_d']:.4f}:{row['c_m']}"
14     return hashlib.sha256(key.encode()).hexdigest()[:16]
15
16 async def load_existing_hashes(conn: asyncpg.Connection,
17                               schema: str) -> dict[str, str]:
18     """Laedt alle bekannten messung_id -> hash Paare aus der DB."""
19     rows = await conn.fetch(

```

```

20         f"SELECT messung_id, content_hash FROM {schema}.messung"
21     )
22     return {r["messung_id"]: r["content_hash"] for r in rows}
23
24 async def delta_sync(
25     df: pd.DataFrame,
26     dsn: str,
27     schema: str = "biofalcon",
28 ) -> dict:
29     """
30     Synchronisiert nur geaenderte/neue Zeilen.
31     Gibt {"new": n, "updated": n, "unchanged": n} zurueck.
32     """
33     df = df.copy()
34     df["content_hash"] = df.apply(row_hash, axis=1)
35
36     conn = await asyncpg.connect(dsn)
37     existing = await load_existing_hashes(conn, schema)
38     await conn.close()
39
40     new_rows = df[~df["messung_id"].isin(existing)]
41     changed_rows = df[
42         df["messung_id"].isin(existing) &
43         (df.apply(
44             lambda r: existing.get(r["messung_id"]) != r["
45                 content_hash"],
46             axis=1
47         ))
48     ]
49     unchanged = len(df) - len(new_rows) - len(changed_rows)
50
51     # Nur Deltas importieren
52     delta = pd.concat([new_rows, changed_rows], ignore_index=True)
53     if len(delta) > 0:
54         from prod_migrate import run_migration, ProdConfig
55         cfg = ProdConfig(dsn=dsn)
56         await run_migration(delta, cfg, mode="upsert")
57
58     return {
59         "new": len(new_rows),
60         "updated": len(changed_rows),
61         "unchanged": unchanged,
62     }

```

Listing 16.9: delta_sync.py: Inkrementelle Synchronisation neuer Messdaten

Kapitel 17

Schluss

Das vorliegende Arbeit hat die vollständige Migration von **Excel + VBA** nach **PostgreSQL + Python** dokumentiert – von der ersten leeren Datenbankinstanz bis zur produktiven, orchestrierten, containerisierten und überwachten Produktivumgebung. Die in dieser Arbeit entwickelten Methoden und Architekturentscheidungen sind jedoch weit mehr als eine punktuelle technische Lösung für ein einzelnes Windkanal-Projekt: Sie repräsentieren einen **Paradigmenwechsel** im Umgang mit strukturierten Daten, der in nahezu allen Sektoren der modernen Wissens- und Industriegesellschaft tiefgreifende Konsequenzen hat. Der folgende Ausblick entfaltet diese Konsequenzen systematisch.

17.1 Technische Weiterentwicklung der Migrationsarchitektur

Die in dieser Arbeit beschriebene Architektur ist bewusst modular gehalten, sodass einzelne Komponenten ohne Rückwirkung auf den Rest ausgetauscht oder erweitert werden können. Folgende Entwicklungslinien sind unmittelbar anschlussfähig:

1. **Change Data Capture (CDC) mit Debezium und Kafka:** PostgreSQL Logical Replication erlaubt es, jede INSERT/UPDATE/DELETE-Operation als Ereignis in einen Apache-Kafka-Topic zu streamen. **debezium** als Kafka-Connector realisiert dies deklarativ ohne Codeänderung am ETL-Skript. Nachgelagerte Systeme (Data Warehouse, ML-Pipeline, Echtzeit-Dashboard) können diesen Stream konsumieren. Gegenüber dem bisherigen Batch-ETL sinkt die Latenz von Stunden auf Millisekunden.
2. **Schema-Migrationen via Alembic:** Das manuelle Nummerierungsschema für SQL-Migrationsskripte lässt sich durch **alembic** ersetzen, das automatisch Upgrade- und Downgrade-Pfade generiert, den aktuellen Schema-Stand in der Datenbank selbst speichert und in Continuous-Integration-Pipelines eingebunden werden kann.
3. **Datenqualitäts-Framework mit Great Expectations:** **great_expectations** ermöglicht die deklarative Definition von Erwartungen an Datensätze (z. B. “Spalte **c_1** liegt stets im Intervall $[-2,5; 3,0]$ ”) und generiert automatisch HTML-Berichte bei Verletzungen.
4. **REST-API-Schicht via FastAPI + asyncpg:** Die asynchrone Datenbankschicht lässt sich durch eine FastAPI-Applikation zu einer vollwertigen REST-API erweitern,

die externen Systemen, mobilen Applikationen und Microservices den Zugriff auf Messdaten ohne direkte Datenbankverbindung ermöglicht.

5. **Machine-Learning-Integration:** Normalisierte PostgreSQL-Daten lassen sich direkt mit `scikit-learn`, `PyTorch` oder `TensorFlow` koppeln. Anomalieerkennung (z. B. Isolation Forest für Ausreißer in Windkanalmessungen) und prädiktive Modelle können vollständig reproduzierbar in die Produktivpipeline integriert werden.
6. **Kubernetes-Deployment mit Helm:** Der Docker-Compose-Stack ist konzeptuell identisch mit einem Kubernetes-Deployment. Via Helm-Charts kann die gesamte Infrastruktur (PostgreSQL, Airflow, Prometheus, Grafana, ETL-Container) auf einem beliebigen Kubernetes-Cluster (on-premise oder Cloud) deployt, skaliert und gerollt werden.

17.2 Konsequenzen für die Wirtschaft und das Management

Die Migration von tabellenkalkulationsbasierter Datenhaltung zu relationalen Datenbanksystemen ist keine rein technische Frage – sie ist eine **strategische Managemententscheidung** mit messbaren ökonomischen Konsequenzen.

17.2.1 Kostensenkung durch Eliminierung von Fehlerquellen

Studien von KPMG, Gartner und PwC beziffern den Anteil von Tabellenkalkulationsfehlern an den Gesamtkosten von Fehlerbereinigungen in Unternehmen auf bis zu 25 % aller operativen Datenfehler. Berühmt-berüchtigte Vorfälle wie die Londoner Whale-Affäre bei JPMorgan Chase (2012), bei der ein Copy-Paste-Fehler in einer `Excel`-Tabelle zur Fehleinschätzung eines Risikos von rund 6,2 Milliarden US-Dollar führte, oder der AstraZeneca-Impfstoff-Fehler (2021), bei dem eine `Excel`-bedingte Datenpanne zu einer Unterlieferung von 17 Millionen Impfdosen führte, verdeutlichen: **das Risiko eines einzelnen unkontrollierten Zellwertes in einer Tabellenkalkulationsdatei kann unternehmerisch existenzbedrohend sein.**

Ein PostgreSQL-basiertes System mit Constraint-Validierung, Transaktionssicherheit und versioniertem ETL-Code eliminiert diese Fehlerklasse strukturell:

- **CHECK**-Constraints verhindern ungültige Werte auf Datenbankebene – nicht nachgelagert in einer Validierungsformel.
- Transaktionen stellen sicher, dass entweder *alle* oder *keine* Daten einer Importoperation geschrieben werden.
- Der Upsert-Mechanismus (`ON CONFLICT DO UPDATE`) verhindert Duplikate strukturell.

17.2.2 Skalierbarkeit als Wettbewerbsvorteil

`Excel`-basierte Prozesse skalieren linear mit dem Personalaufwand: Ein Analyst bearbeitet eine Datei – mehr Daten erfordern mehr Analysten oder längere Arbeitszeiten. Ein PostgreSQL-Backend mit orchestrierter ETL-Pipeline skaliert mit der Hardware: Neue Maschinen werden in den Kubernetes-Cluster aufgenommen, der Airflow-DAG verarbeitet

mehr Daten ohne zusätzliches Personal. Für wachsende Unternehmen, die ihren Datenumfang ver-10-fachen, ist der Wechsel von Excel nach PostgreSQL nicht optional, sondern **Voraussetzung des Wachstums**.

17.2.3 Reduktion von Knowledge-Lock-in

Excel-basierte Wissensspeicher sind häufig an einzelne Mitarbeitende gebunden: Wer die Makros entwickelt hat, versteht sie; wer das Unternehmen verlässt, nimmt das implizite Wissen mit. Versionierter, dokumentierter Python-Code in einem Git-Repository ist hingegen transferierbar, onboardable und wartbar – auch von Mitarbeitenden, die nach dem initialen Entwickler eingestellt werden. Studien zeigen, dass Onboarding-Zeit in datenbankgestützten Umgebungen typischerweise um 30–50 % geringer ist als in Excel-zentrischen Umgebungen.

17.3 Konsequenzen für die Industrieproduktion und Fertigung

In der industriellen Fertigung sind Excel-basierte Prozesse allgegenwärtig: Qualitätsprüfprotokolle, Maschinenkalibrierungsdaten, Wartungsprotokolle, Materialstücklisten (BOMs), Produktionsplanungstabellen. Die Konsequenzen der Migration auf strukturierte Datenbanksysteme sind hier besonders weitreichend.

17.3.1 Echtzeit-Qualitätssicherung (SPC)

Statistical Process Control (SPC) – das kontinuierliche Überwachen von Fertigungsprozessen anhand statistischer Kennzahlen (Mittelwert, Standardabweichung, C_p , C_{pk}) – ist in Excel nur retrospektiv realisierbar. In einem PostgreSQL-basierten System werden Maschinendaten via MQTT-Broker oder OPC-UA direkt in die Datenbank gestreamt; Trigger und materialisierte Views berechnen SPC-Kennzahlen in Echtzeit; Grafana-Dashboards visualisieren Abweichungen sofort. **Ausschuss wird verhindert, nicht berichtet.**

17.3.2 Rückverfolgbarkeit (Traceability) in der normkonformen Fertigung

ISO 9001, IATF 16949 (Automobilindustrie), AS 9100 (Luft- und Raumfahrt) und IEC 62443 (Industrielle Automation) fordern lückenlose Rückverfolgbarkeit jedes gefertigten Teils von der Rohmaterialcharge bis zum Endprodukt. Mit Excel ist diese Traceability nur mit erheblichem manuellem Aufwand herstellbar und bleibt fehleranfällig. Ein normalisiertes PostgreSQL-Schema mit Fremdschlüsseln, Audit-Triggern und Git-versioniertem ETL-Code erfüllt diese Anforderungen strukturell und macht Compliance-Audits nachweisbar statt behauptbar.

17.3.3 Predictive Maintenance und IIoT-Integration

Die vierte industrielle Revolution (Industrie 4.0) erzeugt pro Fertigungsanlage täglich Gigabytes von Sensordaten. Excel ist für diese Datenmengen strukturell ungeeignet.

PostgreSQL mit TimescaleDB-Extension (Zeitreihendaten) in Kombination mit Python-basierten ML-Modellen für Predictive Maintenance ermöglicht es, Maschinenausfälle Stunden oder Tage im Voraus zu prognostizieren. Wartungskosten sinken nachweislich um 25–30 % (McKinsey, 2015), wenn reaktive durch prädiktive Wartung ersetzt wird.

17.4 Konsequenzen für das Finanzwesen

Der Finanzsektor ist besonders exponiert gegenüber den Risiken Excel-basierter Datenhaltung – und gleichzeitig derjenige Sektor, in dem die Vorteile der Migration am unmittelbarsten monetarisierbar sind.

17.4.1 Regulatorische Compliance und Basel III/IV

Die Regularien des Baseler Ausschusses (Basel III und IV), MiFID II, Solvency II (Versicherungen) und DORA (Digital Operational Resilience Act, EU 2022) stellen explizite Anforderungen an Datenqualität, Auditierbarkeit und Systemresilienz von Finanzinstitutionen. Excel-basierte Risikosysteme erfüllen diese Anforderungen strukturell nicht: Sie bieten keine granulare Zugriffskontrolle, keinen lückenlosen Audit Trail und keine Transaktionssicherheit. PostgreSQL mit Row-Level Security, vollständig protokollierten Triggern und Git-versioniertem Quellcode hingegen ist **regulatorisch auditierbar und revisionssicher**.

Eine Studie der European Banking Authority (EBA, 2021) stellte fest, dass über 60 % der gemeldeten Meldewesen-Fehler (COREP, FINREP) auf manuelle Tabellenkalkulationsprozesse zurückzuführen waren. Die Migration auf strukturierte Systeme ist damit nicht nur eine technische Verbesserung, sondern eine **regulatorische Notwendigkeit**.

17.4.2 Risikomanagement und Real-Time-Pricing

Handelsabteilungen großer Banken und Hedgefonds bewerten Portfolios in Echtzeit. Ein Excel-basiertes Bewertungsmodell kann niemals Echtzeit-Marktdaten direkt konsumieren; es wird immer einen manuellen Import- oder Export-Schritt geben. Ein PostgreSQL-Backend mit asyncpg-basiertem Python-Client und direkter Anbindung an Marktdaten-Feeds (Bloomberg, Reuters, OpenBB) ermöglicht Neuberechnungen im Millisekundenbereich. **Latenz im Risikomanagement kostet Geld – jede Sekunde.**

17.4.3 Automatisierte Berichterstattung und Regulatorik

Monatliche, vierteljährliche und jährliche Regulierungsberichte (COREP, FINREP, XBRL-Einreichungen an die EZB) werden in vielen mittelgroßen Instituten noch zu wesentlichen Teilen manuell in Excel aufbereitet. Eine PostgreSQL-basierte Lösung mit versionierten SQL-Abfragen und einer Template-Engine (Jinja2 + LaTeX oder ReportLab) erzeugt diese Berichte vollautomatisch, reproduzierbar und auditierbar – in der Qualität, die Aufsichtsbehörden verlangen.

17.5 Konsequenzen für die wissenschaftliche Forschung

Die Reproduzierbarkeitskrise (*Replication Crisis*) der Wissenschaft – ein breit diskutiertes Phänomen in Psychologie, Biomedizin, Wirtschaftswissenschaften und der Ingenieurforschung – hat eine oft genannte strukturelle Ursache: **nicht reproduzierbare Datenverarbeitungsprozesse**, zu einem erheblichen Teil bedingt durch **Excel**-Dateien als primäre Datenhaltung.

17.5.1 Reproduzierbarkeit als Wissenschaftsnorm (FAIR-Prinzip)

Die FAIR-Prinzipien (Findable, Accessible, Interoperable, Reusable), formuliert von Wilkinson et al. (2016) und seitdem von DFG, EU Horizon und der Nature-Gruppe als Standard gefordert, verlangen, dass Forschungsdaten auffindbar, zugänglich, interoperabel und wiederverwendbar sind. **Excel**-Dateien erfüllen keines dieser Kriterien zuverlässig: Sie sind nicht schema-beschrieben, nicht selbstdokumentierend, nicht maschinenlesbar ohne proprietäre Software und nicht versioniert.

Ein mit Git versioniertes **PostgreSQL**-Schema mit dokumentiertem **Python**-ETL-Code und automatisierten Tests erfüllt alle FAIR-Kriterien:

- **Findable:** DOI über Zenodo, persistenter Datenbankdump.
- **Accessible:** REST-API oder direkte SQL-Abfrage.
- **Interoperable:** Standardisiertes SQL-Schema, CSV/Parquet-Export, maschinenlesbare Metadaten.
- **Reusable:** Vollständiger ETL-Quellcode in Git, Apache-lizenziert, mit Testabdeckung.

17.5.2 Kollaborative Forschung und Open Science

Wissenschaftliche Kollaborationen erstrecken sich über Kontinente. Eine **Excel**-Datei, die per E-Mail in sieben verschiedenen Versionen zirkuliert, ist keine Grundlage für reproduzierbare gemeinsame Forschung. Ein gemeinsames Git-Repository mit **PostgreSQL**-Backups auf einem institutionellen Server oder in der Cloud ermöglicht:

- Gleichzeitigen Lesezugriff beliebig vieler Forscher ohne Dateisperrkonflikte.
- Kontrollierten Schreibzugriff via Pull Requests und Code Review.
- Vollständige Nachvollziehbarkeit, wer welche Messdaten wann eingefügt, korrigiert oder entfernt hat.
- Automatisierte Reproduzierbarkeits-Checks via CI/CD bei jeder Daten- oder Code-Änderung.

17.5.3 Meta-Analysen, Datenfusion und Forschung

Interdisziplinäre Projekte kombinieren Datensätze verschiedener Provenienz: Windkanalmessungen mit CFD-Simulationen, klinische Studien mit genomischen Daten, ökonomische Zeitreihen mit Klimadaten. In einer **Excel**-Welt erfordert jede Fusion manuellen Aufwand zum Abgleich unterschiedlicher Formate, Einheiten und Identifikatoren. Ein normalisiertes Datenbankschema mit klar definierten Fremdschlüsseln macht diese Fusion zu einer SQL-JOIN-Operation.

17.5.4 Langzeitarchivierung von Forschungsdaten

Förderorganisationen wie die DFG, die EU-Kommission (Horizon Europe) und der ERC verlangen, dass Forschungsdaten für mindestens 10, in der Biomedizin häufig 15–20 Jahre nach Projektende verfügbar und lesbar bleiben. **Excel**-Dateien – insbesondere mit **VBA**-Makros – sind durch Formatänderungen von Microsoft nicht zwangsläufig über Jahrzehnte lesbar. Ein **SQL**-Dump eines **PostgreSQL**-Schemas hingegen ist ein offenes Format, das mit jedem Standard-**SQL**-kompatiblen System gelesen werden kann – heute, in 10 Jahren und in 20 Jahren.

17.6 Konsequenzen für das Bildungswesen und die Lehre

Die Diskrepanz zwischen der Allgegenwart von **Excel** in der beruflichen Praxis und den Anforderungen moderner Datenkompetenz (*Data Literacy*) ist einer der zentralen Spannungspunkte in der Hochschullehre.

17.6.1 Datenkompetenz als Kernkompetenz des 21. Jahrhunderts

Das World Economic Forum (WEF) listet in seinem *Future of Jobs Report 2023* “Data Analysis and Science” auf Platz 3 der gefragtesten Kompetenzen der nächsten fünf Jahre. Diese Kompetenz beginnt nicht in **Excel**, sondern mit dem Verständnis strukturierter Datenhaltung, Abfrageformulierung (**SQL**) und programmatischer Datenverarbeitung (**Python**). Hochschulen und Berufsschulen, die ihren Lehrplan auf “wie erstelle ich eine Pivot-Tabelle” ausrichten, produzieren Absolventinnen und Absolventen, die für die Datenwirtschaft des Jahres 2030 nicht vorbereitet sind.

17.6.2 Strukturierte Lehre mit Git und Continuous Integration

Die in dieser Arbeit beschriebenen Werkzeuge – **Git**, **PostgreSQL**, **Python**, **Docker**, **pytest** – sind identisch mit den Werkzeugen moderner Softwareentwicklungskurse. Die Integration von **ETL**-Übungen, Datenbankdesign und Versionskontrolle in ingenieurwissenschaftliche Curricula bietet unmittelbaren Transfer:

- **Laborberichte als Git-Repositories:** Messdaten werden als Pull Request in ein institutionelles Repository eingereicht statt als **.xlsx**-E-Mail-Anhang.
- **Reproduzierbarkeit als Benotungskriterium:** Ein Laborbericht, dessen Ergebnisse nicht reproduzierbar sind (weil der **ETL**-Code fehlt oder nicht läuft), kann konsequent als unvollständig gewertet werden.
- **Peer Review als Lernprinzip:** Code-Reviews im Rahmen von Pull Requests schulen kritisches Lesen fremden Codes – eine Kompetenz, die in reinen **Excel**-Kursen völlig fehlt.

17.7 Konsequenzen für das Gesundheitswesen und die Medizin

Kein Sektor trägt höhere Konsequenzen für Datenfehler als das Gesundheitswesen. Die COVID-19-Pandemie hat die strukturellen Schwächen **Excel**-basierter Datenprozesse in

der öffentlichen Gesundheitsversorgung auf spektakuläre Weise offengelegt: Im Oktober 2020 verlor Public Health England aufgrund eines **Excel**-Zeilenlimits ($2^{20} = 1.048.576$ Zeilen) rund 15.841 COVID-19-positive Fälle aus dem Meldesystem, die daraufhin nicht kontaktverfolgt wurden.

17.7.1 Klinische Studien und GCP-Compliance

Good Clinical Practice (GCP, ICH E6) und 21 CFR Part 11 (FDA, USA) verlangen für klinische Studien eine vollständige, manipulationssichere Dokumentation aller Datenpunkte – inklusive des vollständigen Audit Trails jeder Änderung (wer, wann, was, warum). **Excel** ist explizit *nicht* CFR-Part-11-konform, da seine Änderungshistorie manipulierbar ist. PostgreSQL mit audit-logged Triggern, Row-Level Security und Git-versioniertem ETL-Code hingegen kann so konfiguriert werden, dass CFR-Part-11-Anforderungen strukturell erfüllt sind.

17.7.2 Epidemiologie, Surveillance, Echtzeit-Pandemiemonitoring

Echtzeit-Seuchensurveillance (wie vom RKI, ECDC und der WHO gefordert) erfordert Systeme, die kontinuierlich Meldedaten aus Hunderten von Quellen aggregieren, validieren und visualisieren. Ein PostgreSQL-basierter Surveillance-Stack mit Airflow-Orchestrierung und Grafana-Dashboards ist diesem Problem strukturell gewachsen – ein nationales **Excel**-Netzwerk ist es nicht.

17.8 Konsequenzen für Behörden und Verwaltung

Die öffentliche Verwaltung in Deutschland und Europa ist strukturell **Excel**-abhängig: Haushaltsplanung, Fördermittelverwaltung, Statistik, Meldesysteme – nahezu alle Prozesse werden in Tabellenkalkulationen geführt.

17.8.1 E-Government und Open Data

Die EU-Richtlinie über offene Daten (PSI-Richtlinie, 2019/1024/EU) verpflichtet Behörden, Verwaltungsdaten in maschinenlesbaren, offenen Formaten bereitzustellen. **.xlsx** ist kein offenes Format; CSV, JSON und SQL hingegen sind plattformunabhängig und werkzeuagnostisch. Eine datenbankgestützte Verwaltungsdateninfrastruktur mit REST-API-Schicht erfüllt die PSI-Richtlinie strukturell und bietet gleichzeitig die Grundlage für Open-Data-Portale (CKAN, uData).

17.8.2 Bekämpfung von Haushaltsbetrug und Korruption

Tabellenkalkulationen, die im E-Mail-Umlaufverfahren kursieren, sind strukturell manipulierbar: Zeilen werden gelöscht, Werte verändert, Formeln überschrieben – ohne dass ein Audit Trail verbleibt. Ein transaktionssicheres Datenbankschema mit PostgreSQL-Audit-Extension (**pgaudit**) und Git-versioniertem Quellcode macht solche Manipulationen strukturell unmöglich – oder zumindest lückenlos nachweisbar.

17.9 Konsequenzen für Nachhaltigkeit und Klimaforschung

Klimaforschung und Nachhaltigkeitsberichterstattung sind auf präzise, reproduzierbare und langfristig archivierte Datensätze angewiesen.

17.9.1 Treibhausgas-Bilanzierung und ESG-Reporting

Die EU-Richtlinie zur Nachhaltigkeitsberichterstattung (CSRD, Corporate Sustainability Reporting Directive, 2022/2464/EU) verpflichtet ab 2024 schrittweise alle größeren Unternehmen, umfassende ESG-Berichte (Environment, Social, Governance) zu erstellen – mit revisionssicherem Prüfpfad. Die meisten Unternehmen erstellen ihre CO₂-Bilanzierung (Scope 1, 2, 3-Emissionen) derzeit in **Excel**. **Das genügt den CSRD-Anforderungen nicht:** Auditoren verlangen strukturierte Herkunftsnachweise für jeden Emissionsdatenpunkt. Ein **PostgreSQL**-basiertes ESG-Datenmanagementsystem mit vollständigem Audit Trail, Versions-Tagging und automatisiertem XBRL-Export ist die strukturell korrekte Antwort.

17.9.2 Klimamodellierung und Ensemble-Berechnungen

Numerische Klimamodelle erzeugen Terabytes an Ausgabedaten (NetCDF-Format, ECMWF ERA5-Reanalyse-Daten). Die Aggregation dieser Daten in **Excel** ist technisch ausgeschlossen. **PostgreSQL** mit PostGIS-Extension (räumliche Daten), TimescaleDB (Zeitreihen) und **dask/xarray** (Python-seitige Verarbeitung großer Arrays) ermöglicht die vollständige Forschungsdaten-Pipeline vom Rohdaten-Download bis zur publizierten Abbildung – reproduzierbar, versioniert und FAIR-konform.

17.10 Gesellschaftliche Dimension: Datensouveränität

Die Konzentration kritischer Datenprozesse in einem proprietären Dateiformat eines einzigen Anbieters (Microsoft) ist nicht nur ein technisches, sondern ein **geopolitisches und wirtschaftspolitisches Problem**.

17.10.1 Vendor-Lock-in und digitale Souveränität

Wer seine Betriebsdaten ausschließlich in **.xlsx**-Dateien verwaltet, ist abhängig von der Preisstrategie, den Lizenzentscheidungen und der Produktpolitik eines einzigen US-amerikanischen Konzerns. **PostgreSQL** ist Open-Source-Software unter der PostgreSQL-Lizenz (ähnlich BSD): kein Vendor-Lock-in, keine Lizenzgebühren, kein Risiko einer plötzlichen Preisänderung oder Produkteinstellung. Die EU-Kommission hat im Rahmen ihrer Digital-Souveränitäts-Initiative Open-Source-Datenbanksysteme explizit als strategische Infrastrukturkomponente eingestuft.

17.10.2 Datenschutz (DSGVO) und Datensicherheit

.xlsx-Dateien bieten keinen granularen Datenschutz: Entweder hat eine Person Zugriff auf die gesamte Datei, oder nicht. Row-Level Security in PostgreSQL ermöglicht es, denselben

Datensatz verschiedenen Benutzern in unterschiedlichem Umfang sichtbar zu machen – DSGVO-konform, auditierbar und ohne Datenhaltungsredundanz. Personenbezogene Daten in einer **Excel**-Datei, die per E-Mail versendet wird, erfüllen die DSGVO-Anforderungen an technische und organisatorische Maßnahmen (Art. 32 DSGVO) strukturell nicht.

17.11 Gesamtfazit: Ein Paradigmenwechsel mit globalem Horizont

Die Migration von **Excel** nach **PostgreSQL+Python+Git** ist in ihrer Gesamtheit mehr als eine technische Maßnahme. Sie repräsentiert einen **epistemologischen Wandel** im Umgang mit Wissen, Daten und Prozessen:

Von implizit zu explizit: Wissen, das früher in Zellfarben, manuellen Tabellenkonventionen und undokumentierten **VBA**-Makros kodiert war, wird in explizitem, testbarem und versioniertem Code ausgedrückt.

Von reaktiv zu prädiktiv: Statt Fehler retrospektiv in **Excel** zu dokumentieren, werden sie prospektiv durch Constraints, Tests und CI-Pipelines verhindert.

Von silo-basiert zu kollaborativ: Wissen wandert aus dem Posteingang einzelner Mitarbeitenden in versionierte, durchsuchbare, kollaborativ pflegbare Repositories.

Von proprietär zu souverän: Abhängigkeit von einem einzelnen Softwareanbieter wird durch Open-Source-Infrastruktur ersetzt, die unabhängig von Lizenzentscheidungen betrieben werden kann.

Von ephemer zu archivierbar: Daten, die in sechs Monaten nicht mehr geöffnet werden können, weil die **Excel**-Version inkompatibel geworden ist, werden durch **SQL**-Dumps und **Python**-Code ersetzt, die in 20 Jahren noch lesbar sind.

Die in dieser Arbeit beschriebene Methodik – exemplarisch am Windkanal-Datensatz des BioFalcon-Projekts entwickelt – ist auf nahezu jeden Bereich menschlicher Datenarbeit übertragbar. Jedes Laborprotokoll, jede Haushaltsliste, jedes Qualitätsformular, jede klinische Studie, jeder Emissionsbericht, der heute in einer **.xlsx**-Datei lebt, ist ein Kandidat für diese Migration. Der technische Aufwand dafür ist, wie diese Arbeit zeigt, beherrschbar. Der strukturelle Gewinn – an Qualität, Sicherheit, Reproduzierbarkeit, Kollaborationsfähigkeit und langfristiger Archivierbarkeit – ist **transformativ**.

Jetzt ist der richtige Zeitpunkt für diesen Schritt. Die Werkzeuge sind reif, die Community groß, die Dokumentation exzellent, und die Kosten marginal im Verhältnis zu den Risiken, die eine unkontrollierte **Excel**-basierte Datenwelt langfristig mit sich trägt.

*“Bad data in, bad decisions out – unabhängig davon,
wie schön die Pivot-Tabelle formatiert ist.”*

17.12 Handlungsempfehlungen

Abschließend werden konkrete, sofort umsetzbare Empfehlungen für Organisationen jeder Größe formuliert:

1. **Inventarisierung aller Excel-basierten Kernprozesse:** Identifiziere alle Prozesse im Unternehmen / der Forschungsgruppe, die auf `.xlsx`-Dateien als primären Datenspeicher angewiesen sind. Bewerte das Risikopotenzial nach Fehleranfälligkeit, Reproduzierbarkeit und regulatorischer Relevanz.
2. **Pilotmigration mit einem nicht-kritischen Datensatz:** Starte mit einem Datensatz mittlerer Komplexität (wie dem BioFalcon-Windkanal-Datensatz), der keine direkten Produktiv- oder Compliance-Auswirkungen hat. Entwickle Schema, ETL-Code und Tests. Erfahre die Werkzeuge, bevor kritische Prozesse migriert werden.
3. **Git-Repositories für alle Code- und Schema-Artefakte:** Bereits morgen sollte kein neues SQL-Skript, kein neuer Python-Code und keine neue Konfigurationsdatei außerhalb eines Git-Repositorys entstehen. Das kostet nichts und setzt keine Migration voraus.
4. **Training und Kulturwandel:** Technische Exzellenz allein genügt nicht. Teams müssen in SQL-Grundlagen, Git-Workflows und Python-Skripting geschult werden. Dieser Wandel erfordert Zeit, Führungsunterstützung und eine Lernkultur, die Fehler als Teil des Migrationsprozesses akzeptiert.
5. **Schrittweise Ablösung nach dem Strangler-Fig-Pattern:** Kritische Legacy-Prozesse werden nicht per Big Bang migriert, sondern schrittweise: Ein neues System übernimmt zunächst nur lesende Zugriffe (Reports), dann schreibende (neue Daten), bis das alte Excel-System vollständig stillgelegt werden kann.
6. **Open-Source-First-Strategie:** PostgreSQL, Python, Git, Docker, Grafana, Airflow – alle in dieser Arbeit verwendeten Werkzeuge sind Open Source. Eine Open-Source-First-Strategie vermeidet Vendor-Lock-in, reduziert Lizenzkosten und fördert aktive Teilhabe an der globalen Entwickler-Community.

Die technischen Grundlagen sind gelegt. Die Konsequenzen sind skizziert. **Der nächste Schritt liegt bei den Organisationen, Teams und Einzelpersonen, die erkennen, dass eine `.xlsx`-Datei mit einem Dateinamen wie `Messung_v7_FINAL_final2_korr.xlsx` kein Zukunftsmodell ist – und handeln.**

Literaturverzeichnis

- [1] McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, 51–56.
- [2] PostgreSQL Global Development Group (2024). *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>
- [3] Bayer, M. (2012). *SQLAlchemy*. The Architecture of Open Source Applications, Vol. II. <https://www.sqlalchemy.org>
- [4] Gazoni, E. & Clark, C. (2023). *openpyxl – A Python library to read/write Excel files*. <https://openpyxl.readthedocs.io>
- [5] Codd, E.F. (1972). *Further Normalization of the Data Base Relational Model*. IBM Research Report RJ909.
- [6] Kimball, R. & Ross, M. (2013). *The Data Warehouse Toolkit*, 3rd ed. Wiley.
- [7] Selivanov, Y. (2019). *asyncpg: A fast PostgreSQL Database Client Library for Python/asyncio*. <https://magicstack.github.io/asyncpg/>
- [8] Apache Software Foundation (2024). *Apache Airflow Documentation*. <https://airflow.apache.org/docs/>
- [9] Prefect Technologies (2024). *Prefect Documentation*. <https://docs.prefect.io/>
- [10] Prometheus Authors (2024). *Prometheus: Monitoring System and Time Series Database*. <https://prometheus.io/docs/>
- [11] Docker Inc. (2024). *Docker Documentation*. <https://docs.docker.com/>
- [12] HashiCorp (2024). *Vault: Secrets Management*. <https://developer.hashicorp.com/vault/docs>
- [13] MacIver, D.R. & Hatfield-Dodds, Z. (2019). *Hypothesis: A new approach to property-based testing*. Journal of Open Source Software, 4(43), 1891.
- [14] Chacon, S. & Straub, B. (2014). *Pro Git*, 2nd ed. Apress. <https://git-scm.com/book>
- [15] Driessen, V. (2010). *A successful Git branching model*. <https://nvie.com/posts/a-successful-git-branching-model/>
- [16] Conventional Commits Authors (2023). *Conventional Commits Specification v1.0.0*. <https://www.conventionalcommits.org/>

- [17] Krekel, H. et al. (2024). *pytest: helps you write better programs*. <https://docs.pytest.org/>
- [18] Astral (2024). *Ruff: An extremely fast Python linter and formatter*. <https://docs.astral.sh/ruff/>
- [19] Lehtosalo, J. et al. (2024). *mypy: Optional Static Typing for Python*. <https://mypy.readthedocs.io/>
- [20] Bayer, M. (2024). *Alembic: Database Migrations for SQLAlchemy*. <https://alembic.sqlalchemy.org/>
- [21] Superconductive Inc. (2024). *Great Expectations: Data Quality for Pipelines*. <https://docs.greatexpectations.io/>
- [22] Ramirez, S. (2024). *FastAPI: Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com/>
- [23] Red Hat / Debezium Authors (2024). *Debezium: Change Data Capture for a variety of databases*. <https://debezium.io/documentation/>
- [24] Apache Software Foundation (2024). *Apache Kafka Documentation*. <https://kafka.apache.org/documentation/>
- [25] Timescale Inc. (2024). *TimescaleDB: Time-series data for PostgreSQL*. <https://docs.timescale.com/>
- [26] PostGIS Development Team (2024). *PostGIS: Spatial and Geographic Objects for PostgreSQL*. <https://postgis.net/documentation/>
- [27] Cloud Native Computing Foundation (2024). *Kubernetes Documentation*. <https://kubernetes.io/docs/>
- [28] Helm Authors (2024). *Helm: The Kubernetes Package Manager*. <https://helm.sh/docs/>
- [29] Grafana Labs (2024). *Grafana Documentation*. <https://grafana.com/docs/grafana/>
- [30] pre-commit Authors (2024). *pre-commit: A framework for managing and maintaining multi-language pre-commit hooks*. <https://pre-commit.com/>
- [31] Wilkinson, M.D. et al. (2016). *The FAIR Guiding Principles for scientific data management and stewardship*. Scientific Data, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- [32] Baker, M. (2016). *1,500 scientists lift the lid on reproducibility*. Nature, 533, 452–454. <https://doi.org/10.1038/533452a>
- [33] Panko, R. R. (2008). *What We Know About Spreadsheet Errors*. Journal of End User Computing, 10(2), 15–21.
- [34] Europäische Kommission (2022). *Richtlinie 2022/2464/EU: Corporate Sustainability Reporting Directive (CSRD)*. Amtsblatt der Europäischen Union, L 322, 15–80.

- [35] Europäisches Parlament und Rat (2019). *Richtlinie 2019/1024/EU über offene Daten und die Weiterverwendung von Informationen des öffentlichen Sektors*. Amtsblatt der Europäischen Union, L 172, 56–83.
- [36] Europäisches Parlament und Rat (2022). *Verordnung (EU) 2022/2554: Digital Operational Resilience Act (DORA)*. Amtsblatt der Europäischen Union, L 333, 1–79.
- [37] Europäisches Parlament und Rat (2016). *Verordnung (EU) 2016/679: Datenschutz-Grundverordnung (DSGVO)*. Amtsblatt der Europäischen Union, L 119, 1–88.
- [38] U. S. Food and Drug Administration (2023). *21 CFR Part 11: Electronic Records; Electronic Signatures*. Code of Federal Regulations, Title 21. <https://www.ecfr.gov/current/title-21/chapter-I/subchapter-A/part-11>
- [39] World Economic Forum (2023). *Future of Jobs Report 2023*. Geneva: World Economic Forum. <https://www.weforum.org/reports/the-future-of-jobs-report-2023/>
- [40] McKinsey Global Institute (2015). *The Internet of Things: Mapping the Value Beyond the Hype*. McKinsey & Company.
- [41] pgAudit Authors (2024). *pgAudit: PostgreSQL Audit Extension*. <https://www.pgaudit.org/>
- [42] Harris, C. R. et al. (2020). *Array programming with NumPy*. Nature, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- [43] Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- [44] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
- [45] Dask Development Team (2024). *Dask: Scalable analytics in Python*. <https://docs.dask.org/>
- [46] Hoyer, S. & Hamman, J. (2017). *xarray: N-D labeled Arrays and Datasets in Python*. Journal of Open Research Software, 5(1), 10. <https://doi.org/10.5334/jors.148>
- [47] CERN & OpenAIRE (2024). *Zenodo: Research. Shared*. <https://zenodo.org/>